Acta Astronautica 249 (2026) 1277–1298

Contents lists available at ScienceDirect

Acta Astronautica

journal homepage: www.elsevier.com/locate/actaastro

Research paper

Gaussian-disturbance-aware deep reinforcement learning for event-triggered MPC in satellite attitude reorientation

Liushuai Zheng [a,b], Yadong Chen [a,b], Zixuan Zhou [a,b], Peng Cheng [a,b,c,*]

[a] School of Aeronautics and Astronautics, Sichuan University, Chengdu 610207, China
[b] Robotic Satellite Key Laboratory of Sichuan Province, Sichuan University, Chengdu 610207, China
[c] National Key Laboratory of Fundamental Science on Synthetic Vision, Sichuan University, Chengdu 610207, China

ARTICLE INFO

Keywords:
Satellite attitude reorientation
Deep reinforcement learning
Event-triggered control
Structured state space model
Kalman filter

ABSTRACT

Satellite attitude reorientation is a critical maneuvering task in space missions. However, the presence of strong nonlinearity, high coupling, and external disturbances poses significant challenges for conventional control approaches. Traditional methods often struggle to balance control accuracy and computational efficiency without compromising robustness. To address these challenges in a systematic manner, this paper proposes a Gaussian-disturbance-aware deep reinforcement learning (DRL) framework for event-triggered model predictive control (ET-MPC) in satellite attitude reorientation, consisting of two key components. First, to handle system nonlinearity while improving resource efficiency, a DRL-based agent learns a MPC triggering policy. In particular, a structured state space model (SSM) is incorporated into the DRL architecture to enhance temporal dependency modeling and policy learning capability. Second, to cope with Gaussian disturbance encountered during on-orbit operations, a Kalman filter (KF) is embedded within the framework. By filtering Gaussian noise in the angular velocity measurements, the KF provides the DRL triggering policy and MPC optimizer with more reliable information, thereby improving closed-loop performance under the measurement-noise conditions considered in this study. Simulation results demonstrate that the proposed framework achieves accurate attitude reorientation while reducing unnecessary MPC updates compared with baseline methods. Moreover, the framework exhibits improved robustness in disturbed environments, indicating its potential for computationally efficient satellite attitude reorientation.

1. Introduction

Satellite attitude reorientation is a fundamental technology for ensuring the successful execution of a wide range of space missions, including Earth observation, deep-space exploration, and inter-satellite communications. The design of attitude controllers has long been confronted with an inherent trade-off between operational efficiency and control robustness [1]. This challenge has been further exacerbated by the growing prevalence of resource-constrained platforms, such as nanosatellites, which have become increasingly prominent in modern space missions [2]. On the one hand, stringent constraints on onboard power consumption, computational capability, and actuator lifespan necessitate highly efficient control strategies to minimize resource usage and extend their operational lifetime in orbit. On the other hand, the physical characteristics of such platforms significantly increase their sensitivity to complex time-varying disturbances, including solar radiation pressure and atmospheric drag, thereby imposing substantial challenges on the control system's ability to maintain reorientation accuracy and post-maneuver attitude stability. Consequently, a critical challenge in contemporary aerospace engineering lies in the development of intelligent control frameworks that effectively reconcile efficiency with robustness, particularly under external disturbances and onboard resource constraints.

In early studies on satellite attitude reorientation, model predictive control (MPC) attracted considerable attention owing to its ability to perform online receding-horizon optimization. Extensive research has been conducted on MPC itself [3–5], as well as on its nonlinear, robust, and other advanced variants [6–10]. Nevertheless, the inherently high computational burden of MPC has severely restricted its practical deployment on resource-constrained platforms. To mitigate this limitation, event-triggered MPC (ET-MPC) schemes were introduced to reduce resource consumption. The core idea of ET-MPC is to recompute the MPC solution and update the control input only when the system state error or a specified performance metric exceeds a predefined

* Corresponding author.
E-mail address: chengpeng@scu.edu.cn (P. Cheng).

https://doi.org/10.1016/j.actaastro.2026.08.022
Received 19 May 2026; Received in revised form 29 July 2026; Accepted 5 August 2026
Available online 10 August 2026
0094-5765/© 2026 IAA. Published by Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

threshold. By avoiding the frequent and often unnecessary updates associated with conventional time-triggered control, ET-MPC effectively alleviates computational and communication loads [11]. Consequently, this approach has attracted growing research interest and inspired a range of application-oriented studies [12–15]. These studies cover a variety of practical scenarios. For flexible spacecraft, an adaptive control strategy incorporating fuzzy logic was developed within the ET-MPC framework to address the nonlinearities arising from rigid–flexible coupling [12]. In the context of space telescope reorientation, an enhanced robust MPC scheme was proposed to balance control performance with actuation efficiency [13]. Another study introduced a distributed MPC approach with an adaptive dual-horizon mechanism, in which the prediction horizons are dynamically adjusted to reduce online computational effort [14]. More recently, a distributed constrained control method based on the ET-MPC paradigm was developed to achieve finite-time spacecraft synchronization [15].

These conventional approaches demonstrate strong performance in their respective application scenarios. However, they depend heavily on accurate system dynamics models. Such reliance inevitably degrades their effectiveness in the presence of unmodeled dynamics and significant uncertainties, thereby making it difficult to satisfy the stringent requirements of complex space missions. To alleviate this limitation, recent studies have introduced neural networks into satellite attitude control frameworks [16–20]. For example, single-critic neural networks have been employed to approximate optimal cost functions for spacecraft formation control within an ET-MPC scheme [16]. In addition, neural networks have been integrated into dual-channel ET-MPC architectures to achieve robust spacecraft formation control under communication constraints [17]. Neural networks have also been embedded in adaptive ET-MPC frameworks to solve robust attitude control problems for uncertain spacecraft with limited communication resources [18]. Another research direction leverages neural networks to extract high-dimensional features, which are subsequently linearized via the deep Koopman operator [19]. Building on this representation, a distributionally robust optimization strategy combined with a tailored ET-MPC framework was developed, resulting in notable improvements in overall control performance [20]. Related work has also investigated neural-network-based relative pose estimation for noncooperative spacecraft, demonstrating the importance of reliable state information for autonomous spacecraft guidance and control [21]. This observation is also consistent with the use of reliable filtered state information in the proposed framework.

Although the aforementioned deep learning approaches have partially alleviated model dependency, their training processes typically remain reliant on offline datasets or predefined objective functions. Owing to the lack of mechanisms for online policy adaptation and real-time environmental feedback, these methods often struggle to achieve truly adaptive optimization of control policies. To facilitate a paradigm shift from passive control responses to proactive decision making, it is essential to develop an intelligent control architecture endowed with online adaptation and optimization capabilities. In this context, deep reinforcement learning (DRL) methods [22–26] offer distinct advantages. Specifically, DRL can learn decision policies through direct interaction with the environment without requiring an explicit analytical triggering law, and it exploits powerful nonlinear function approximation to handle complex decision-making problems. The integration of DRL with ET-MPC has already demonstrated promising results in challenging application domains such as autonomous driving [27–29], robotics [30–32], and resource scheduling [33–35], highlighting its potential for addressing complex control problems. DRL has also been applied to autonomous space tasks, including active tracking of noncooperative space objects and motion planning for space robotic systems [36–38]. Nevertheless, in the context of satellite attitude control, research efforts exploring DRL-based approaches and their variants remain relatively limited [39–41]. Existing studies have followed several research directions. An improved Deep Deterministic Policy Gradient (DDPG) algorithm was proposed by incorporating neural-network-based environment modeling and simulated annealing to accelerate exploration, thereby enhancing convergence speed and control accuracy [39]. Subsequently, controllers based on DDPG and Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm were developed for unconstrained, state-constrained, and online optimization problems [40]. Further advances introduced a Multi-Agent DDPG (MADDPG) framework for collision-free trajectory planning during satellite formation reconfiguration, achieving fast and stable trajectory generation while satisfying safety and actuator constraints [41]. Collectively, these studies confirm the potential of DRL for satellite attitude control. However, they generally fail to fully consider a critical practical scenario, namely the simultaneous presence of stringent onboard resource constraints and external disturbances. As a result, the adaptability and practicality of existing DRL-based methods in real-world aerospace applications remain open challenges.

To address the research gap identified above, this paper proposes a Gaussian-disturbance-aware DRL-based ET-MPC framework for satellite attitude reorientation. The primary objective is to learn when the MPC optimization problem should be re-solved during the reorientation maneuver, thereby reducing unnecessary MPC updates and onboard computational burden while maintaining reorientation accuracy and robustness against external disturbances. The central scientific challenge lies in achieving a synergistic trade-off between MPC update sparsity and closed-loop control performance under disturbed and resource-constrained conditions. The main contributions of this work are summarized as follows:

(1) A DRL-based ET-MPC framework is developed for satellite attitude reorientation. Unlike conventional ET-MPC approaches [12–15], the proposed framework exploits the online decision-making capability of DRL to learn when the MPC optimization problem should be re-solved during the reorientation maneuver. As a result, the control system can dynamically balance reorientation performance and computational resource efficiency under varying operational conditions.

(2) Compared with existing learning-based ET-MPC frameworks [16–19], this work incorporates a Mamba-based SSM architecture into the DRL triggering policy. This design enhances temporal dependency modeling by capturing the evolution of attitude error, angular velocity response, control input, and triggering history during the reorientation process. By exploiting these temporal features, the proposed method improves the policy's ability to evaluate whether the cached MPC control sequence remains effective.

(3) To enhance robustness under Gaussian disturbance conditions, Kalman filter (KF) is further integrated into the proposed DRL-based triggering architecture. In contrast to existing reinforcement learning applications in satellite attitude control [39–41], the proposed design provides filtered angular velocity information, together with the normalized measured quaternion, to the DRL triggering policy and MPC optimizer, thereby reducing the influence of Gaussian disturbances on the state information used for ET-MPC triggering decisions and improving the robustness of the overall framework.

(4) Comparative simulations demonstrate that the proposed algorithm outperforms benchmark DRL methods [22–26] and a representative event-triggered learning-based approach [28]. The results indicate that the proposed method reduces unnecessary MPC updates while maintaining accurate attitude reorientation performance under Gaussian disturbance conditions, thereby achieving a favorable trade-off between computational efficiency and control performance.

The remainder of this paper is organized as follows. Section 2 presents the system model and problem formulation. Section 3 describes the proposed algorithmic framework in detail. Section 4 evaluates the performance of the proposed method through comprehensive comparative experiments. Finally, Section 5 concludes the paper and outlines directions for future research.

Notations: $\mathbb{R}$ and $\mathbb{N}$ denote the sets of real numbers and natural numbers, respectively. The semidefinite ordering of matrices is denoted by $\preccurlyeq$. I_n represents the n-dimensional identity matrix. For a matrix A, A^T and $\mathrm{tr}(A)$ denote its transpose and trace, respectively. The Euclidean norm is denoted by $\|\cdot\|_2$. For a vector $a = [a_x, a_y, a_z]^{\mathrm{T}} \in \mathbb{R}^3$, its cross-product operator $a^{\times}$ is defined by the skew-symmetric matrix as: $a^{\times} = [0, -a_z, a_y; a_z, 0, -a_x; -a_y, a_x, 0]$. A unit quaternion is denoted by $q = [q_s, q_v^{\mathrm{T}}]^{\mathrm{T}} = [q_s, q_x, q_y, q_z]^{\mathrm{T}} \in \mathbb{R}^4$, where $q_s \in \mathbb{R}$ represents the scalar part and $q_v = [q_x, q_y, q_z]^{\mathrm{T}} \in \mathbb{R}^3$ represents the vector part. The unit-norm constraint is satisfied, i.e., $q_s^2 + q_x^2 + q_y^2 + q_z^2 = 1$. The conjugate of q is defined as $q^* = [q_s, -q_v^{\mathrm{T}}]^{\mathrm{T}}$. In the MPC formulation, X_k and U_k denote the state and control input sequences, respectively. The predicted state at time $k+i$, based on information available at time k, is denoted by $x_{i|k}$.

2. System modeling and problem formulation

2.1. Satellite attitude control system model

The Euler angle representation is widely used for attitude parameterization. However, owing to its inherent singularity issues, the unit quaternion representation is adopted in this paper to describe the attitude of the satellite body frame with respect to the inertial reference frame. The corresponding attitude kinematic and dynamic equations of the satellite are given as:

$$\begin{cases} \dot{q}_s = -\dfrac{1}{2} q_v^{\mathrm{T}} \omega \\ \dot{q}_v = \dfrac{1}{2}\left(q_s \omega + q_v^{\times} \omega\right) \\ J\dot{\omega} = -\omega^{\times} J\omega + u + d \end{cases} \tag{1}$$

where $J \in \mathbb{R}^{3\times3}$ denotes the inertia matrix, $\omega = [\omega_x, \omega_y, \omega_z]^{\mathrm{T}} \in \mathbb{R}^3$ represents the angular velocity, $u = [u_x, u_y, u_z]^{\mathrm{T}} \in \mathbb{R}^3$ denotes the control torque, and $d \in \mathbb{R}^3$ represents the external disturbance.

The desired attitude is described by the unit quaternion $q_d = [q_{ds}, q_{dv}^{\mathrm{T}}]^{\mathrm{T}}$, and the desired angular velocity is denoted by ω_d. The attitude and angular velocity errors are defined as:

$$\begin{cases} q_e = q_d^* \otimes q \\ \omega_e = \omega - C\omega_d \end{cases} \tag{2}$$

where $q_e = [q_{es}, q_{ev}^{\mathrm{T}}]^{\mathrm{T}} \in \mathbb{R}^4$ denotes the attitude error quaternion, and $\omega_e = [\omega_{ex}, \omega_{ey}, \omega_{ez}]^{\mathrm{T}} \in \mathbb{R}^3$ denotes the angular velocity error. The rotation matrix $C \in \mathbb{R}^{3\times3}$ associated with q_e is given by:

$$C = (q_{es}^2 - q_{ev}^{\mathrm{T}} q_{ev}) I_3 + 2 q_{ev} q_{ev}^{\mathrm{T}} - 2 q_{es} q_{ev}^{\times} \tag{3}$$

Based on the above definitions, the attitude error kinematics and the angular velocity error dynamics can be written as:

$$\begin{cases} \dot{q}_{es} = -\dfrac{1}{2} q_{ev}^{\mathrm{T}} \omega_e \\ \dot{q}_{ev} = \dfrac{1}{2}\left(q_{es}\omega_e + q_{ev}^{\times}\omega_e\right) \\ J\dot{\omega}_e = -\omega^{\times} J\omega - J\left(C\dot{\omega}_d - \omega_e^{\times} C\omega_d\right) + u + d \end{cases} \tag{4}$$

2.2. Problem formulation

This paper considers a nonlinear discrete-time system of the following form:

$$x_{k+1} = F(x_k, u_k, d_k) \tag{5}$$

where $x_k = [q_k^{\mathrm{T}}, \omega_k^{\mathrm{T}}]^{\mathrm{T}} \in \mathbb{R}^7$ denotes the system state and $u_k \in \mathbb{R}^3$ represents the control torque at time step k. The nonlinear mapping $F(\cdot)$ describes the discrete-time rigid-body dynamics of the satellite, obtained by numerically integrating the continuous-time equations of motion over one sampling interval using a fourth-order Runge–Kutta scheme. The disturbance term $d_k \in \mathbb{R}^3$ represents an external disturbance acting on the angular velocity dynamics. The proposed approach is formulated within the MPC framework. At each triggering instant k, the controller solves the following finite-horizon optimal control problem (OCP) using IPOPT through MPCTools:

$$\min_{X_k, U_k} \quad J_{\mathrm{mpc}} = \sum_{i=0}^{p-1} \ell\left(x_{i|k}, u_{i|k}\right)$$

$$\text{s.t.} \quad x_{0|k} = \hat{x}_k \tag{6}$$

$$x_{i+1|k} = F(x_{i|k}, u_{i|k}, d_{i|k}), \quad i = 0, \dots, p-1 \tag{7}$$

$$x_{\min} \le x_{i|k} \le x_{\max}, \quad i = 0, \dots, p \tag{8}$$

$$u_{\min} \le u_{i|k} \le u_{\max}, \quad i = 0, \dots, p-1 \tag{9}$$

$$\Delta u_{\min} \le u_{0|k} - u_{k-1} \le \Delta u_{\max} \tag{10}$$

$$\Delta u_{\min} \le u_{i+1|k} - u_{i|k} \le \Delta u_{\max}, \quad i = 0, \dots, p-2 \tag{11}$$

$$d_{\min} \le d_{i|k} \le d_{\max}, \quad i = 0, \dots, p-1 \tag{12}$$

where p denotes the prediction horizon. $\hat{x}_k = [(q_k^m)^{\mathrm{T}}, \hat{\omega}_{k|k}^{\mathrm{T}}]^{\mathrm{T}}$ denotes the composite state supplied to MPC, where q_k^m is the normalized quaternion measurement and $\hat{\omega}_{k|k}$ is the posterior angular velocity estimate generated by the KF. The stage cost function $\ell(x_{i|k}, u_{i|k})$ penalizes the attitude error, angular velocity error and control effort at each prediction step. The predicted state sequence is denoted by $X_k = [x_{0|k}, x_{1|k}, \dots, x_{p|k}]$, and the predicted control sequence by $U_k = [u_{0|k}, u_{1|k}, \dots, u_{p-1|k}]$. In the MPC prediction, the admissible disturbance sequence $\{d_{i|k}\}_{i=0}^{p-1}$ is considered within the compact set $[d_{\min}, d_{\max}]$, enabling robust constraint handling in the presence of external perturbations.

No separate terminal cost or terminal constraint is included in the current optimal control problem. The controller uses a finite-horizon stage cost and updates the MPC solution when an event is triggered. This design avoids introducing additional terminal ingredients and allows the present study to focus on the learning-based MPC triggering mechanism. A rigorous stability analysis of the current control framework is beyond the scope of this study. Without a terminal condition and with the short prediction horizon of $p = 5$, the effectiveness of the cached control sequence may be affected, which may in turn influence the timing and frequency of MPC re-optimization. The overall closed-loop control performance and triggering behavior under this design are evaluated through the numerical simulations presented in Section 4.

2.3. DRL-ET-MPC framework

This section introduces the DRL-ET-MPC framework, which utilizes DRL to learn an ET-MPC triggering strategy. At each time step k, the actor network outputs a continuous action $a_k \in \mathbb{R}$. A binary trigger signal $\delta_k \in \{0, 1\}$ is then generated based on a_k as follows:

$$\delta_k = \mathbb{I}_{\{|a_k| > \eta\}} \tag{13}$$

where $\eta \ge 0$ is a predefined event-trigger threshold, and $\mathbb{I}_{\{\cdot\}}$ denotes the indicator function. Although η is fixed, the trigger signal is state-dependent because a_k is generated by the learned DRL policy. Therefore, the agent learns when the continuous triggering action should exceed the threshold to invoke MPC re-optimization.

If $\delta_k = 1$, the controller solves the MPC problem to obtain a new optimal control sequence U_k^* and applies the first control input $u_{0|k}^*$. If $\delta_k = 0$, MPC re-optimization is skipped, and the controller advances through the previously cached MPC control sequence. Once the final element of the cached sequence is reached, the final cached input is held until a subsequent learned trigger invokes MPC re-optimization.

The stage cost is defined as:

$$\ell(x, u) = w_q \|q_{ev}\|_2^2 + w_\omega \|\omega_e\|_2^2 + w_u \|u\|_2^2 \tag{14}$$

where $w_q = 10$, $w_\omega = 0.4$, and $w_u = 10^{-3}$ are the weights assigned to the attitude error, angular velocity error, and control effort,

respectively. The attitude error term is assigned the largest weight because accurate attitude reorientation is the primary control objective. The angular velocity term suppresses residual rotational motion, while the smaller control-effort weight discourages excessive actuator use without limiting the necessary control action.

The MPC objective over a prediction horizon p is given by:

$$J_{\mathrm{mpc}}(X_k, U_k) = \sum_{i=0}^{p-1} \ell\left(x_{i|k}, u_{i|k}\right) \tag{15}$$

Using the trigger signal δ_k, the performance of the learned policy is evaluated via the cumulative return over an episode of length K:

$$R = \sum_{k=0}^{K-1} r_k = -\sum_{k=0}^{K-1} \left(\ell(x_k, u_k)\Delta t + \lambda \delta_k\right) \tag{16}$$

where u_k denotes the control input actually applied to the satellite at time step k, $\lambda \geq 0$ is a penalty weight that trades off control performance against triggering frequency, and Δt represents the sampling interval. This reward signal r_k encourages the DRL agent to achieve accurate attitude reorientation while minimizing unnecessary MPC computations.

The event-trigger frequency is defined as:

$$f_t = \frac{1}{K} \sum_{k=0}^{K-1} \delta_k \tag{17}$$

which quantifies the fraction of time steps at which MPC re-optimization is invoked over the entire episode, but does not reveal the temporal distribution of these invocations.

To provide this temporal information, the cumulative trigger count up to time step k is defined as:

$$N_{\mathrm{tr}}(k) = \sum_{i=0}^{k} \delta_i, k = 0, \ldots, K-1 \tag{18}$$

where $N_{\mathrm{tr}}(k)$ records the total number of triggering events up to and including time step k.

3. Proposed Gaussian-disturbance-aware ET-MPC framework

The proposed framework is designed to address three coupled challenges in DRL-based ET-MPC. First, the triggering decision should be generated from a learned continuous action rather than being determined solely by a manually designed state-error triggering rule. Second, whether the cached MPC control sequence remains effective cannot be reliably determined from an instantaneous observation alone, since it depends on the recent evolution of attitude errors, angular velocity responses, and previous triggering decisions. Third, in disturbed environments, raw measurements may lead to unreliable triggering decisions and inaccurate MPC initialization. To address these issues in a unified framework, the KF first provides filtered angular velocity estimates for both the triggering policy and the MPC optimizer. A Mamba-based temporal encoder embedded in the TD3 actor–critic networks then extracts temporal features from the recent state evolution to assess whether the cached MPC control sequence remains effective. Based on these temporal features, TD3 serves as a high-level triggering policy that determines whether MPC re-optimization is required, whereas MPC remains the low-level controller responsible for generating the constraint-aware control sequence. Thus, the KF improves state-information reliability, Mamba provides temporal context, TD3 determines the MPC update timing, and MPC performs predictive control under system and actuator constraints.

3.1. TD3-based ET-MPC triggering policy

To formulate the ET-MPC policy optimization problem within a DRL framework, this paper first constructs a Markov decision process (MDP) with continuous state and action spaces, represented by the tuple (S, A, P, R, γ). Here, S denotes the state space observed by the agent, and A represents the action space, consisting of real-valued inputs over a continuous interval. P describes the state transition probability distribution $P(s' \mid s, a)$, which defines the probability of transitioning from the current state $s \in S$ to the next state $s' \in S$ after taking action $a \in A$. R is the reward function designed to balance control performance and resource consumption; and $\gamma \in (0, 1]$ is the discount factor that balances immediate and long-term cumulative rewards. It should be emphasized that TD3 is not used here as a direct attitude controller. Instead, it serves as a high-level triggering policy that determines whether the MPC optimization problem should be re-solved at the current time step. The low-level control input is still generated by MPC or reused from the cached MPC sequence. This hierarchical design preserves the constraint-handling capability of MPC while allowing the DRL agent to learn the resource–performance trade-off in the event-triggering process.

In the proposed triggering MDP, the state s_k is constructed from the estimated attitude-motion information and its recent temporal context, the action a_k is a scalar continuous triggering action, and the binary triggering signal δ_k is obtained by thresholding a_k. After δ_k is determined, the applied control u_k is either computed by solving the MPC problem or obtained by reusing the cached MPC input sequence. Therefore, the reward is designed to penalize not only attitude reorientation errors and control effort, but also unnecessary MPC re-optimizations.

TD3 is adopted for three reasons that are directly related to ET-MPC triggering. First, the triggering decision is represented by a continuous action before being converted into a binary event signal, which makes deterministic continuous-control learning suitable for this problem. Second, the off-policy replay mechanism allows the agent to reuse transition data collected under different triggering patterns, improving the efficiency of policy learning. Third, the clipped double-Q mechanism and delayed policy update help reduce unstable policy oscillations, which is particularly important because frequent fluctuations of the triggering policy may cause unnecessary MPC re-optimizations.

The schematic of the TD3-based triggering policy optimization is shown in Fig. 1. In it, $\{(s_j, a_j, r_j, s'_j)\}_{j=1}^{N}$ denotes a batch of transitions sampled from the experience replay buffer; π_ϕ and $\pi_{\phi'}$ represent the actor network and its corresponding target network, respectively, while Q_{θ_i} and $Q_{\theta'_i}$ denote the ith critic network and its target network ($i = 1, 2$).

The critic parameters $\theta_i (i = 1, 2)$ are updated by minimizing the mean squared error:

$$\mathcal{L}(\theta_i) = \frac{1}{N} \sum_{j=1}^{N} \left(Q_{\theta_i}(s_j, a_j) - y_j\right)^2 \tag{19}$$

where the target value y_j is computed using the twin target critic networks with clipped noise:

$$\begin{aligned} y_j &= r_j + \gamma \min_{i=1,2} Q_{\theta'_i}\left(s'_j, \pi_{\phi'}(s'_j) + \epsilon\right) \\ \epsilon &\sim \mathrm{clip}\left(\mathcal{N}(0, \sigma), -c, c\right) \end{aligned} \tag{20}$$

where γ is the discount factor, the min operation mitigates overestimation bias and the clipped noise ϵ is added for target policy smoothing.

The actor parameter ϕ is updated to maximize the expected Q-value, following the deterministic policy gradient computed using the first critic network Q_{θ_1}:

$$\nabla_\phi J(\phi) = \frac{1}{N} \sum_{j=1}^{N} \nabla_a Q_{\theta_1}(s_j, a)\Big|_{a=\pi_\phi(s_j)} \nabla_\phi \pi_\phi(s_j) \tag{21}$$

The target networks are updated via exponential moving averages to stabilize training:

$$\begin{aligned} \phi' &\leftarrow \tau\phi + (1-\tau)\phi' \\ \theta'_1 &\leftarrow \tau\theta_1 + (1-\tau)\theta'_1 \\ \theta'_2 &\leftarrow \tau\theta_2 + (1-\tau)\theta'_2 \end{aligned} \tag{22}$$

where $\tau \in (0, 1)$ is the soft update rate.

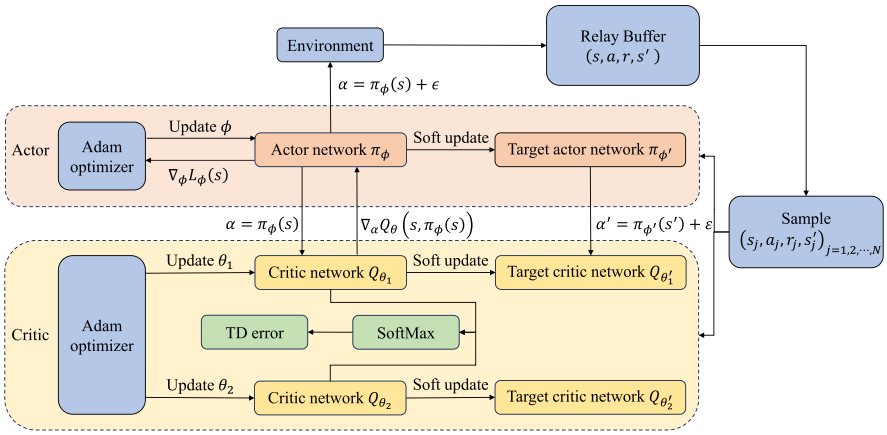


Fig. 1. TD3-based triggering policy optimization for ET-MPC.

3.2. Mamba-enhanced temporal triggering policy

In the TD3-based ET-MPC triggering framework, the key decision is not to directly compute the control input, but to determine whether the previously optimized MPC control sequence remains effective for the current and near-future system evolution. This decision is inherently temporal. For example, a transient attitude error, a slowly accumulating angular velocity deviation, and a persistent measurement fluctuations may require different triggering responses even when their instantaneous state magnitudes are similar. Therefore, relying only on a memoryless multilayer perceptron (MLP) representation may lead to myopic triggering decisions, because the agent cannot fully infer whether the cached MPC sequence is still reliable from a single-step observation.

To address this issue, a Mamba-based temporal encoder is incorporated into the TD3 actor–critic architecture, leading to the proposed MTD3 algorithm. The purpose of introducing Mamba [42] is not merely to replace the MLP backbone, but to provide the triggering policy with trajectory-level information for assessing the effectiveness of the cached MPC control sequence. By extracting temporal features from recent attitude-motion evolution, the agent can determine whether the current MPC sequence can continue to be reused or whether re-optimization is necessary.

Compared with Transformer-based architectures [43], the Mamba sequence model provides an efficient alternative for temporal representation learning. It is based on the structured state space model (SSM) and employs an input-dependent selective mechanism to capture long-range dependencies with linear computational complexity. This property is particularly suitable for ET-MPC triggering, where the policy needs to process recent closed-loop evolution efficiently while avoiding excessive additional onboard computational burden.

Recent studies have also applied Mamba-based state space models to reinforcement learning. Decision Mamba develops a multi-grained SSM with self-evolution regularization for offline reinforcement learning [44], whereas MA-Mamba employs a multi-channel Mamba framework to model temporal information and agent interactions in cooperative multi-agent reinforcement learning [45]. Unlike these offline and multi-agent settings, the present work uses Mamba as a temporal encoder within a single-agent TD3-based triggering policy to assess the effectiveness of the cached MPC control sequence and determine the MPC update timing. The schematic of the MTD3 framework is shown in Fig. 2. In this framework, both the actor and critic networks are equipped with a Mamba trajectory encoder. The input to the encoder is a temporal state sequence constructed from recent estimated attitude-motion states. Specifically, a sequence tensor is formed as $\mathbf{s}_{\text{batch}} \in \mathbb{R}^{B\times L\times N}$, where B denotes the batch size, L represents the total sequence length, and N indicates the state dimension at each time step. Each element of the sequence corresponds to the estimated attitude-motion state at a recent time step. In this way, the encoder captures the temporal evolution of attitude and angular velocity states, which are the most relevant features for determining whether MPC re-optimization is necessary.

The encoder first maps each time step of the input sequence into a d-dimensional feature space through a linear projection layer. The resulting feature sequence is then processed by a Mamba block. Through the selective SSM mechanism, the encoder adaptively retains informative temporal patterns and suppresses redundant or short-lived fluctuations. After root mean square (RMS) normalization, average pooling over the temporal dimension is used to compress the sequence into a fixed-dimensional latent representation $\mathbf{h}_{\text{batch}} \in \mathbb{R}^{B\times d}$. This latent representation summarizes the recent closed-loop evolution of the satellite attitude system and is subsequently used for triggering decision making.

For the actor network, the Mamba encoder maps the temporal state sequence s_k into a latent feature h_k, which is then passed through an MLP head to generate the continuous triggering action. For the critic network, the same type of Mamba encoder is used to extract the temporal feature h_k from the state sequence. The extracted feature is concatenated with the triggering action a_k to form a joint state–action representation, which is then fed into an MLP head to estimate the Q-value. In this way, the critic evaluates not only the instantaneous triggering action, but also the temporal context under which the action is taken.

The temporal modeling capability of MTD3 is particularly important for reducing unnecessary MPC re-optimizations. When the recent trajectory indicates that the attitude error and angular velocity response are converging smoothly, the actor tends to output sub-threshold triggering actions, allowing the controller to reuse the cached MPC sequence. Conversely, when the temporal features indicate a potential degradation of tracking performance or a degradation in the effectiveness of the cached sequence, the actor outputs a triggering action that exceeds the threshold, thereby activating MPC re-optimization. Thus, MTD3 transforms the event-triggering mechanism from a memoryless threshold detector into a temporal triggering policy that evaluates the validity of the MPC control sequence based on recent closed-loop behavior.

3.3. KF-enhanced Gaussian-disturbance-aware triggering framework

Although the MTD3 algorithm can exploit temporal information to reduce unnecessary MPC re-optimizations, its triggering decision may still be affected by Gaussian disturbances in the angular-velocity measurements. In practical satellite attitude reorientation tasks, sensor noise may cause the measured angular velocity to deviate from its true value. If the DRL triggering policy directly uses such disturbed observations, it may misinterpret measurement noise as genuine state evolution, resulting in unnecessary MPC re-optimization or inappropriate reuse of the cached MPC control sequence.

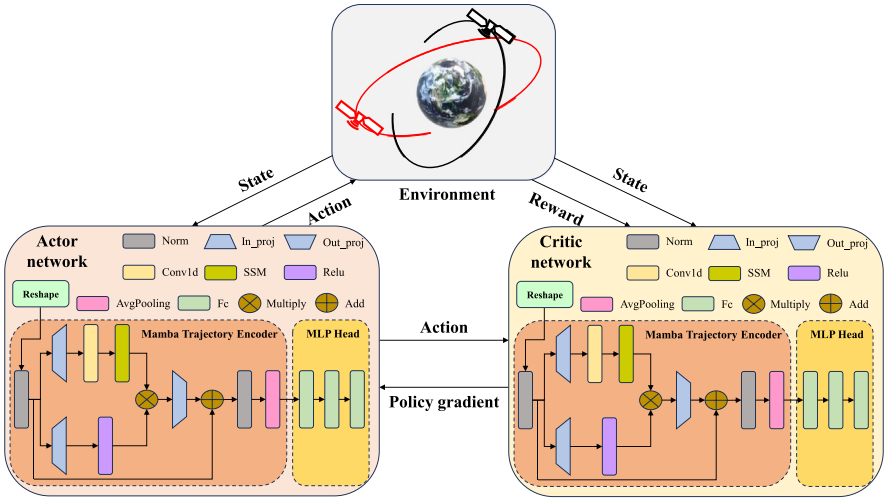


Fig. 2. Mamba-enhanced TD3 architecture for temporal ET-MPC triggering.

To mitigate this problem, the KF [46] is introduced into the proposed framework. The KF is not used as an independent compensation module. Instead, it serves as a shared state-estimation front-end for both the DRL triggering policy and the MPC optimizer. The posterior angular-velocity estimate is combined with the normalized quaternion measurement to construct the state information supplied to the KMTD3 agent and the MPC optimizer. Therefore, the triggering decision and the subsequent MPC solution are based on consistent state information.

The role of the KF can be understood from the perspective of Gaussian-disturbance-aware triggering. Rather than feeding raw disturbed measurements directly into the policy network, the KF recursively combines model-based prediction and noisy measurement correction to generate a more reliable posterior angular-velocity estimate. The estimation-error covariance is used internally by the KF to compute the Kalman gain, which adaptively balances the trust placed on the model prediction and the measurement update. In this sense, the proposed triggering framework is Gaussian-disturbance-aware because the angular-velocity component of the state representation used for decision making is shaped by covariance-weighted estimation rather than by raw noisy observations.

In the present implementation, the KF is applied only to the angular velocity rather than to the full quaternion-angular-velocity state. In the numerical simulations, the quaternion measurement is assumed to be noise-free and is normalized before being supplied to the controller. Therefore, the quaternion is not included in the KF state.

The standard KF was selected because the estimation task in this work is restricted to the three-dimensional angular-velocity channel with additive Gaussian measurement noise, while the normalized quaternion is assumed to be directly available. Under this reduced-order formulation, the prediction and measurement models used by the filter are linear in the estimated state, providing a transparent and computationally efficient estimation front end consistent with the onboard-resource considerations of the proposed framework. Since the quaternion is not included in the KF state, the nonlinear quaternion propagation and unit-norm preservation issues motivating the MEKF do not arise in the present implementation. Nevertheless, MEKF, UKF, or CKF would be more appropriate for applications involving noisy quaternion measurements or joint nonlinear estimation of attitude and angular velocity [47,48].

The noisy measurement of angular velocity is denoted as $\boldsymbol{z}_k \in \mathbb{R}^3$. The reduced-order model used by the KF is given by:

$$\boldsymbol{\omega}_{k+1} = A_\omega \boldsymbol{\omega}_k + B_\omega \boldsymbol{u}_k + \boldsymbol{\nu}_k \tag{23}$$

$$\boldsymbol{z}_k = H_\omega \boldsymbol{\omega}_k + \boldsymbol{\nu}_k \tag{24}$$

where $A_\omega = I_3$, $B_\omega = \Delta t J_f^{-1}$, $H_\omega = I_3$ with J_f denoting the inertia matrix used in the reduced-order KF prediction model. $\boldsymbol{\nu}_k \sim \mathcal{N}(0, Q)$ and $\boldsymbol{\nu}_k \sim \mathcal{N}(0, R)$ denote the effective process-model uncertainty and angular-velocity measurement noise, respectively. The matrices Q and R denote the corresponding covariance matrices assumed by the KF, respectively. The gyroscopic coupling term omitted from the reduced-order prediction model, together with discretization residuals, is treated as process-model uncertainty. The KF prediction and correction equations are as follows:

$$\hat{\boldsymbol{\omega}}_{k+1|k} = A_\omega \hat{\boldsymbol{\omega}}_{k|k} + B_\omega \boldsymbol{u}_k \tag{25}$$

$$P_{k+1|k} = A_\omega P_{k|k} A_\omega^{\mathrm{T}} + Q \tag{26}$$

$$K_{k+1} = P_{k+1|k} H_\omega^{\mathrm{T}} \left(H_\omega P_{k+1|k} H_\omega^{\mathrm{T}} + R \right)^{-1} \tag{27}$$

$$\hat{\boldsymbol{\omega}}_{k+1|k+1} = \hat{\boldsymbol{\omega}}_{k+1|k} + K_{k+1} \left(\boldsymbol{z}_{k+1} - H_\omega \hat{\boldsymbol{\omega}}_{k+1|k} \right) \tag{28}$$

$$P_{k+1|k+1} = \left(I_3 - K_{k+1} H_\omega \right) P_{k+1|k} \tag{29}$$

Based on the above design, the proposed KMTD3 algorithm integrates the KF with the Mamba-enhanced TD3 triggering policy. As illustrated in Fig. 3, at each time step k, the KF receives the noisy angular-velocity measurement $\boldsymbol{z}_k$ and the previous control input $\boldsymbol{u}_{k-1}$, and produces the posterior angular-velocity estimate $\hat{\boldsymbol{\omega}}_{k|k}$. This estimate is combined with the normalized quaternion measurement to form the current composite state estimate $\hat{\boldsymbol{x}}_k$. The resulting composite state estimate serves two purposes. First, it is assigned to the MPC initial state $\boldsymbol{x}_{0|k}$ whenever replanning is triggered. Second, it is used to construct the temporal input of the KMTD3 agent.

Specifically, the state sequence used by the KMTD3 agent at step k is formed as a sliding window:

$$s_k = \left[\hat{x}_{k-L+2}, \ldots, \hat{x}_k, \ \hat{x}_k^{\mathrm{cache}} \right] \tag{30}$$

where $\hat{x}_k^{\mathrm{cache}}$ denotes a model-based prediction feature derived from the most recent valid cached MPC solution. If such a recent MPC solution is available, this term is taken as the MPC-predicted state $\hat{x}_{1|k-1}^{\mathrm{MPC}}$. Otherwise, $\hat{x}_k^{\mathrm{cache}}$ defaults to the current composite state estimate $\hat{x}_k$.

This construction directly links the temporal state representation with the event-triggering objective. The recent composite-state estimates $\mathcal{X}_k = [\hat{x}_{k-L+2}, \ldots, \hat{x}_k]$ describe the recent estimated evolution of the satellite attitude system. Therefore, the agent does not merely

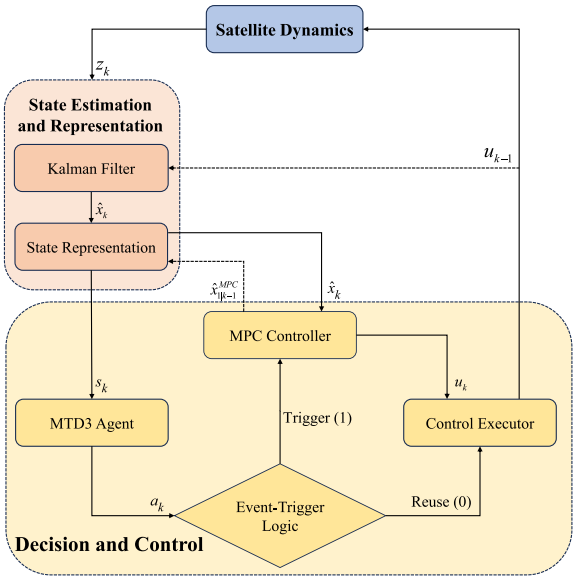


Fig. 3. KMTD3 framework schematic diagram.

observe a sequence of past states; instead, it receives a compact temporal representation that helps assess whether the cached MPC sequence remains reliable for continued reuse. A large inconsistency between the recent estimated evolution and the MPC-predicted state may indicate that re-optimization is necessary.

The proposed KMTD3 framework forms a closed information loop among estimation, event triggering, and constrained optimization. The KF converts noisy angular-velocity measurements into posterior angular-velocity estimates. The Mamba-based TD3 triggering policy evaluates the recent estimated trajectory together with the MPC-predicted state and decides whether the cached MPC sequence should be reused. Once replanning is triggered, MPC uses the resulting composite state estimate as its initial condition and generates a control sequence under system constraints. Therefore, the four modules operate at different functional levels: the KF improves angular-velocity estimation reliability, Mamba extracts temporal features for assessing the validity of the cached MPC sequence, TD3 determines the MPC update timing, and MPC performs predictive optimization while explicitly accounting for system and actuator constraints. The framework is thus functionally coupled rather than a parallel stacking of independent modules.

The pseudocode of KMTD3 algorithm is summarized as follows.

Algorithm 1 KMTD3 Algorithm

Input: Training steps T, batch size B, other hyperparameters
Output: Trained policy parameter ϕ

Initialize actor network π_ϕ and critic networks $Q_{\theta_1}, Q_{\theta_2}$ with Mamba encoders
Initialize target networks $\pi'_\phi \leftarrow \pi_\phi, Q'_{\theta_1} \leftarrow Q_{\theta_1}, Q'_{\theta_2} \leftarrow Q_{\theta_2}$
Initialize replay buffer D, total steps $\leftarrow 0$
Reset environment and construct initial state sequence s_0 using $\hat{x}_{0|0}$
$s_k \leftarrow s_0$
for $k = 0$ to $T-1$ **do**
  Increment total steps: total steps $\leftarrow$ total steps + 1
  if total steps < start timesteps **then**
    Select random action a_k uniformly from $[a_{\min}, a_{\max}]$
  else
    Select action a_k using the policy network
    Add exploration noise ϵ: $a_k \leftarrow \text{clip}(a_k + \epsilon, a_{\min}, a_{\max})$
  end if
  Generate binary trigger signal δ_k from a_k using event-trigger threshold η
  if $\delta_k = 1$ **then**
    Solve OCP using current KF estimate $\hat{x}_k$ to obtain control sequence U_k and predicted state sequence X_k
    Cache U_k in the Control Executor and reset its internal index
  else
    Instruct the Control Executor to increment its internal index
  end if
  Obtain final control command u_k from the Control Executor
  Evolve plant dynamics with u_k and obtain disturbed measurement z_{k+1}
  Update the KF using u_k and z_{k+1} to obtain $\hat{\omega}_{k+1|k+1}$, and combine it with the normalized quaternion measurement to construct $\hat{x}_{k+1}$
  Construct next state sequence s_{k+1} according to (30)
  Compute reward r_k
  done $\leftarrow$ CheckTerminal($\hat{x}_{k+1}, k$)
  Store transition $(s_k, a_k, r_k, s_{k+1}, \text{done})$ in D
  $s_k \leftarrow s_{k+1}$
  if $|D| \geq B$ **and** total steps $\geq$ start timesteps **then**
    Sample $\{(s_j, a_j, r_j, s_{j+1})\}_{j=1}^{N}$ from D
    Compute target value y_j in (20)
    Update critic networks in (19)
    if total steps $\mod d = 0$ **then**
      Update actor network in (21)
      Update target networks in (22)
    end if
  end if
end for

4. Simulation results and analysis

This section first presents the simulation setup, including the experimental configuration and evaluation metrics. The proposed data-driven algorithm framework for satellite attitude reorientation is then assessed through two types of numerical simulations. To address the double-cover property of unit quaternions, where both q and $-q$ represent the same physical attitude, two distinct initial conditions are considered. Specifically, $q_{es} > 0$ corresponds to Simulation Example 1, while $q_{es} < 0$ corresponds to Simulation Example 2. This setup enables the evaluation of the algorithm's robustness and convergence under both quaternion representations.

4.1. Simulation settings

Numerical simulations are conducted to validate the proposed algorithm's effectiveness and superiority. To reduce the influence of random variations, each experiment is repeated five times using different random seeds, and the results are reported as the mean ± standard deviation in the corresponding tables. The simulations are executed in a PyCharm environment using Python, the CUDA 12.1 computing platform, and Windows 11 operating system. The hardware setup includes a 12th Gen Intel® Core™ i7-12700 K CPU (3.60 GHz) and an NVIDIA GeForce RTX 3090 Ti GPU with 24 GB of memory. Detailed parameter configurations of the algorithm are provided in Table 1.

4.2. Evaluation metrics

In the following experiments, the proposed algorithm is compared with other algorithms using six key evaluation metrics for satellite attitude reorientation: total cost (J_{mpc}), episode return (R), event-trigger frequency (f_t), cumulative trigger count (N_{tr}), mean attitude error

Table 1
Parameter settings.

Parameter	Value
Batch size	64
Gamma	0.99
Buffer size	1×10^4
Policy noise	0.2
Noise clip	0.5
Policy frequency	2
Actor learning rate	5×10^{-4}
Critic learning rate	5×10^{-4}
Hidden sizes	[256, 256]
Event-trigger threshold	0.5
Start timesteps	5000
Total training steps	50 000
Weight decay	5×10^{-6}
Prediction horizon	5
Nominal Inertia Matrix	diag(1,1,1)
KF prediction Inertia Matrix	diag(0.95,1.05,0.98)
Angular velocity range	$[-\pi, \pi]^3$
Control torque range	$[-5, 5]^3$
Runge–Kutta integration step size	0.2
Mamba sequence length	10
Mamba block number	1
Mamba model dimension	64
OCP convergence tolerance	10^{-8}
KF process covariance	$7 \times 10^{-6} I_3$
KF assumed measurement covariance	$1 \times 10^{-2} I_3$
KF initial error covariance	$1 \times 10^{-2} I_3$

(MAE), and mean angular velocity error (MAVE). The first four metrics are calculated as shown in (15), (16), (17), and (18), respectively. MAE and MAVE are computed as follows:

$$\mathrm{MAE} = \frac{1}{K} \sum_{k=0}^{K-1} \left\| \mathbf{q}_{ev,k} \right\|_2 \tag{31}$$

$$\mathrm{MAVE} = \frac{1}{K} \sum_{k=0}^{K-1} \left\| \boldsymbol{\omega}_{e,k} \right\|_2 \tag{32}$$

As defined in (16), the episode return R is the cumulative sum of per-step rewards, with each reward being the negative of the total instantaneous cost. This cost consists of the stage cost $\ell(x, u)$ and the event-trigger penalty $\lambda \delta_k$. The stage cost penalizes attitude error, angular velocity error, and control energy consumption, while the event-trigger penalty discourages unnecessary MPC re-optimizations by penalizing event-trigger actions. Consequently, a higher return R indicates better overall algorithm performance in terms of reorientation accuracy, energy efficiency, and event-trigger frequency.

Specifically, smaller MAE and MAVE indicate smaller average deviations from the desired attitude and zero angular velocity, respectively. Lower control effort generally indicates reduced actuator demand and energy consumption. Meanwhile, a lower event-trigger frequency implies that the MPC is activated only when necessary, significantly reducing computational burden and communication overhead. Therefore, a high-performance attitude reorientation algorithm should optimize both control energy consumption and triggering frequency while ensuring high precision.

To evaluate the performance of KMTD3 algorithm, it is compared with a traditional threshold control method (denoted as Threshold), several DRL methods (including DQN, DDQN, and DDPG) and a deep reinforcement learning method (denoted as DPL in this study) from autonomous driving field. Specifically, the DPL combines DDQN, prioritized experience replay, and an LSTM layer to learn binary MPC triggering decisions. And it is adapted to the satellite attitude reorientation setting using the same control environment and evaluation criteria as the other comparison methods.

4.3. Simulation Example 1

In Simulation Example 1, the initial state is defined by the attitude quaternion $q_0 = [0.9660, -0.0690, 0.1380, -0.2070]^{\mathrm{T}}$ and the angular velocity $\omega_0 = [0.4, -0.3, 0.2]^{\mathrm{T}}$ rad/s. Independent Gaussian disturbances drawn from $\mathcal{N}(0, 0.05^2)$ are added to the three angular velocity measurement channels. The desired attitude is $q_d = [0.7070, 0.1890, -0.3780, 0.5670]^{\mathrm{T}}$, and the desired angular velocity is $\omega_d = [0, 0, 0]^{\mathrm{T}}$ rad/s. For this configuration, the scalar part of the error quaternion satisfies $q_{es} > 0$, indicating that the minimum rotation angle required to align the current attitude with the target attitude is less than 180°. This scenario corresponds to typical attitude corrections and high-precision pointing tasks for satellites in orbit, which are common in conventional attitude adjustment applications.

Under multi-seed conditions, the detailed results over five random seeds are summarized in Table 2. The table shows that, while most DRL methods reduce the event-trigger frequency compared to the Threshold approach, these improvements often come at the cost of higher total cost and larger mean tracking errors. For instance, DDPG achieves a 34.9% reduction in event-trigger frequency compared to Threshold; however, its total cost, mean attitude error, and mean angular velocity error increase by 8%, 15.4%, and 0.6%, respectively. In contrast, the KMTD3 algorithm significantly outperforms these comparison methods across all evaluation metrics. Compared with Threshold, DQN, DDQN, DPL, DDPG, and TD3, KMTD3 reduces the total cost by 6.7%, 9.4%, 9.1%, 13.3%, 13.7%, and 19.8%, respectively. The corresponding reductions in MAVE are 56.0%, 56.8%, 53.0%, 57.6%, 56.3%, and 59.1%, while those in event-trigger frequency are 77.8%, 71.6%, 67.7%, 66.7%, 65.9%, and 68.4%, respectively. In terms of the cumulative trigger count, KMTD3 records an average of only 8.4 triggers over the simulation horizon, substantially fewer than the 37.8 triggers of Threshold and the 24.6–29.6 triggers of the comparison DRL methods. These results demonstrate that KMTD3 not only overcomes the over-triggering issue inherent in the traditional threshold method but also effectively mitigates the common trade-off between accuracy and efficiency seen in existing DRL approaches. Therefore, KMTD3 simultaneously achieves high control precision and a reduced MPC invocation burden.

Fig. 4 illustrates the event-trigger sequences of the compared algorithms in a single run of Simulation Example 1. The KMTD3 algorithm activates MPC re-optimization only when necessary, resulting in sparse and efficient triggering behavior. Its total number of triggers is significantly lower than that of all baselines, thereby substantially reducing the number of MPC re-optimizations while highlighting its adaptive responsiveness to satellite attitude dynamics. Fig. 5 depicts the temporal distribution of trigger events throughout the simulation. Although the baseline curves contain some flat intervals influenced by the method, these curves continue to accumulate trigger events throughout most of the maneuver. In contrast, KMTD3 primarily triggered during the initial adjustment phase and remained largely inactive thereafter, exhibiting only a few isolated responses. This pattern indicates that KMTD3 provides active regulation during the transient phase while avoiding unnecessary updates once the system has stabilized. Figs. 6 and 7 show the time evolution of attitude error and angular velocity error for a representative trial, respectively. Under Gaussian noise in the angular velocity measurements, all algorithms exhibit more pronounced and frequent fluctuations in mean angular velocity error than in mean attitude error. Nevertheless, KMTD3 demonstrates the best overall performance, with both errors converging smoothly without overshoot or chattering to the lowest steady-state values. This stable convergence clearly outperforms the oscillatory responses characteristic of other methods. Fig. 8 displays the control torque profiles of each algorithm in a single run of this simulation. During the transient phase, KMTD3 exhibits stable adjustment, with its steady-state output virtually free of chattering. In contrast, the baseline methods experience pronounced high-frequency oscillations. This smooth torque profile

Table 2
Detailed experimental results in Simulation Example 1.

Metric	Threshold	DQN	DDQN	DPL	DDPG	TD3	KMTD3
R	−7.033	−6.777	−6.577	−6.790	−6.787	−7.308	**−5.217**
	± 0.287	± 0.264	± 0.070	± 0.075	± 0.144	± 0.390	± **0.191**
J_{mpc}	5.144	5.297	5.277	5.530	5.557	5.978	**4.797**
	± 0.064	± 0.216	± 0.105	± 0.131	± 0.341	± 0.512	± **0.241**
MAE	0.052	0.053	0.054	0.058	0.060	0.066	**0.046**
	± 0.002	± 0.005	± 0.003	± 0.004	± 0.008	± 0.009	± **0.008**
MAVE	0.159	0.162	0.149	0.165	0.160	0.171	**0.070**
	± 0.006	± 0.013	± 0.009	± 0.014	± 0.009	± 0.006	± **0.003**
f_t	0.189	0.148	0.130	0.126	0.123	0.133	**0.042**
	± 0.023	± 0.012	± 0.013	± 0.013	± 0.024	± 0.020	± **0.008**
N_{tr}	37.800	29.600	26.000	25.200	24.600	26.600	**8.400**
	± 4.658	± 2.408	± 2.646	± 2.588	± 4.827	± 3.975	± **1.673**

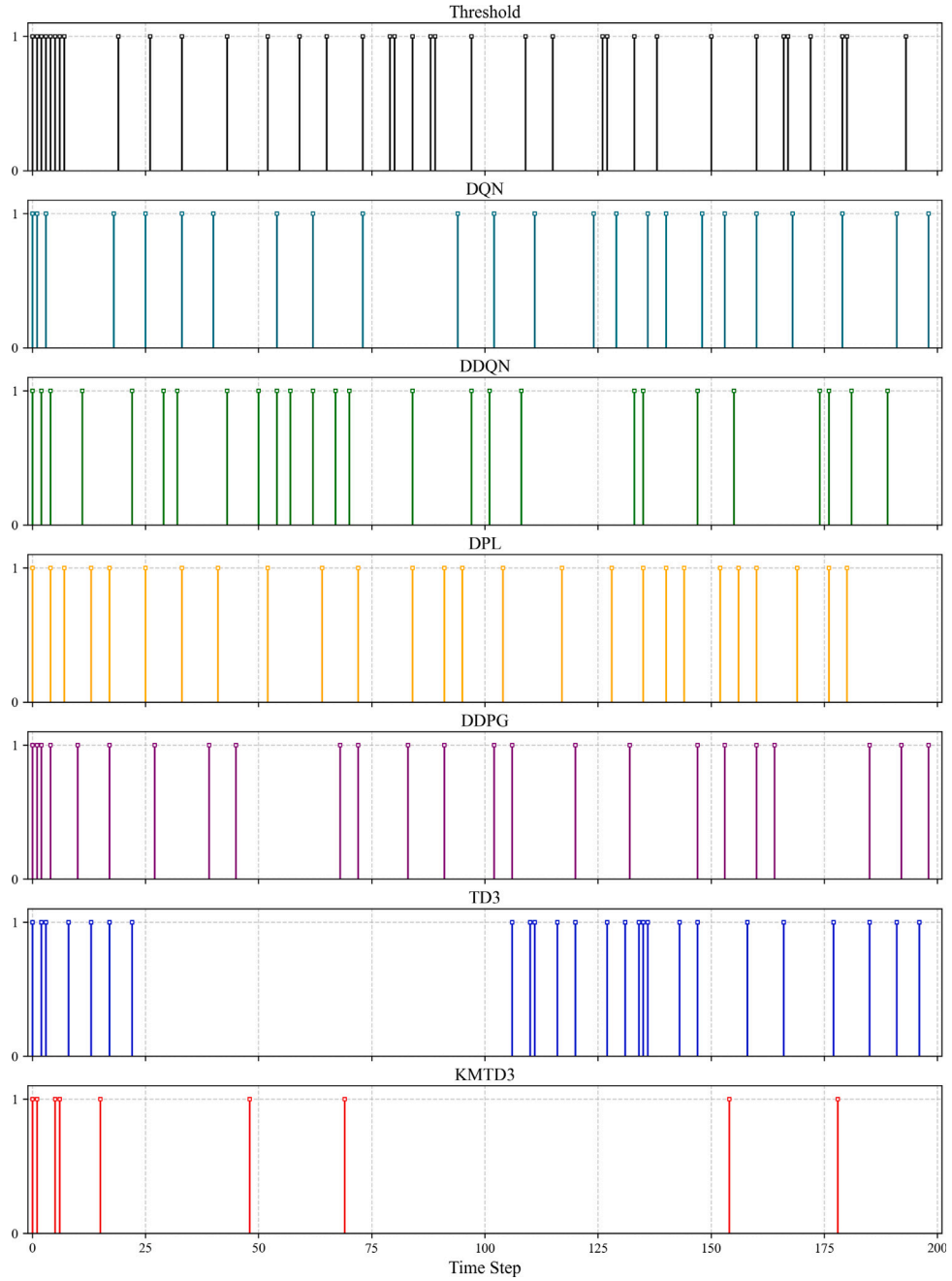


Fig. 4. Comparison of event-trigger frequency in Simulation Example 1.

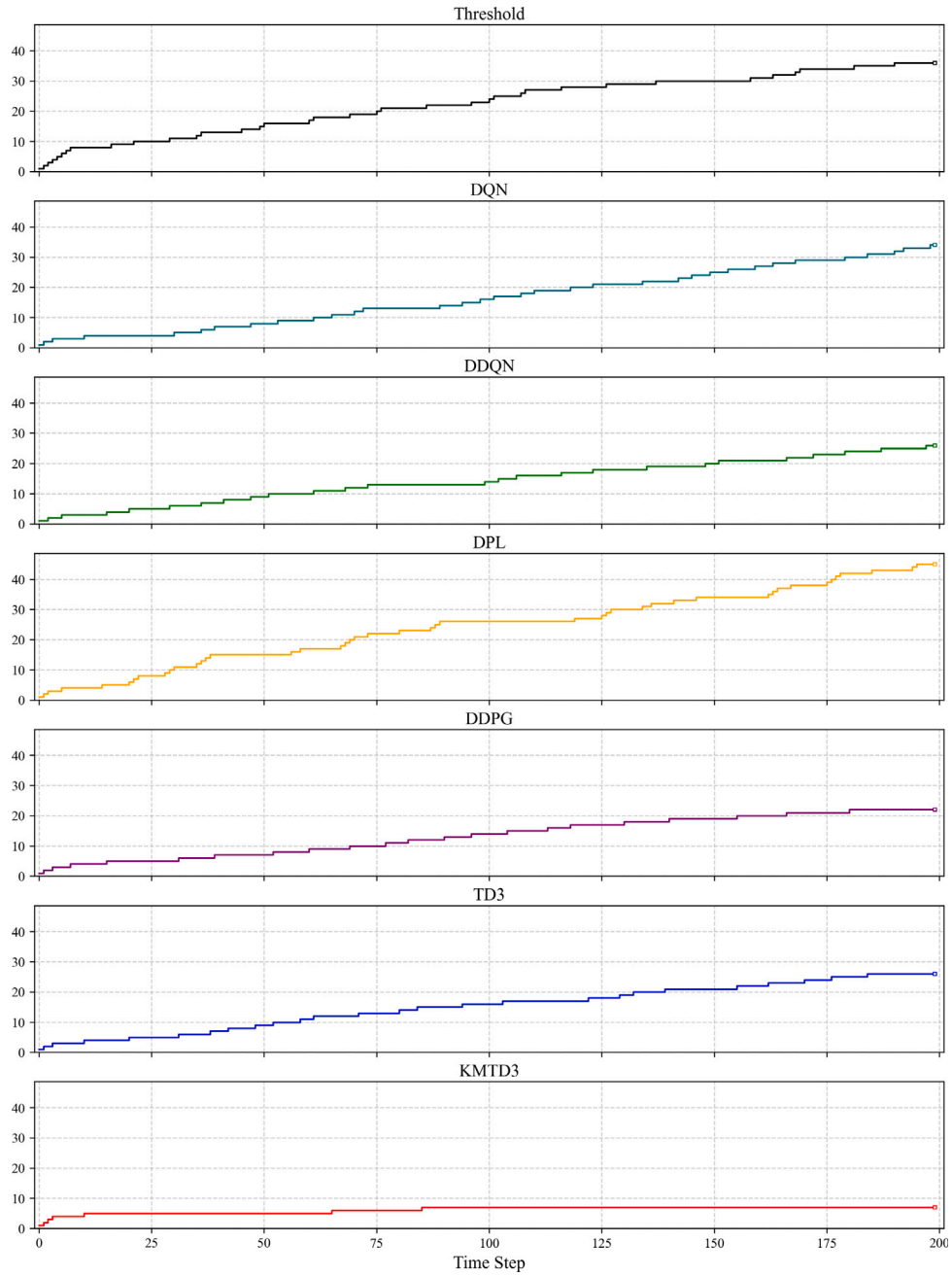


Fig. 5. Comparison of cumulative trigger counts in Simulation Example 1.

not only reduces actuator energy consumption and mechanical wear but also highlights the engineering practicality of the proposed strategy. Specifically, KMTD3 effectively balances control accuracy, system stability, and onboard resource efficiency under disturbance, meeting the stringent reliability requirements of satellite attitude reorientation missions.

4.4. Simulation Example 2

Unlike the previous example, the initial attitude in Simulation Example 2 is represented by quaternion $q_0 = [0.9400, -0.0690, 0.1380, -0.3043]^{\mathrm{T}}$, and the initial angular velocity is $\omega_0 = [0.4, -0.3, 0.2]^{\mathrm{T}}$ rad/s. The same Gaussian disturbance as in Simulation Example 1 is applied. The desired target attitude is $q_d = [-0.7500, 0.1890, 0.3780, -0.5088]^{\mathrm{T}}$, and the desired angular velocity is $\omega_d = [0, 0, 0]^{\mathrm{T}}$ rad/s. For this configuration, the scalar part of the error quaternion satisfies $q_{es} < 0$, corresponding to a negative-scalar quaternion representation. Owing to the double-cover property, q_e and $-q_e$ represent the same physical attitude error, and the minimum rotation angle remains $2\arccos(|q_{es}|) \leq 180°$. This configuration is used to evaluate the proposed method under a large-angle attitude mismatch represented by a negative-scalar error quaternion.

As in Simulation Example 1, the proposed KMTD3 algorithm is compared with multiple algorithms in Simulation Example 2, and the quantitative results over five random seeds are summarized in Table 3. Despite the challenging large-angle initial condition and negative-scalar error quaternion representation, KMTD3 consistently outperforms both the traditional threshold method and existing DRL approaches across all evaluation metrics. Specifically, KMTD3 achieves the lowest total cost of 4.759, representing reductions of 6.3%, 9.1%, 8.2%, 11.5%, 16.0%, and 15.0% compared to Threshold, DQN, DDQN, DPL, DDPG, and TD3, respectively. It also attains the highest episode return of −5.219,

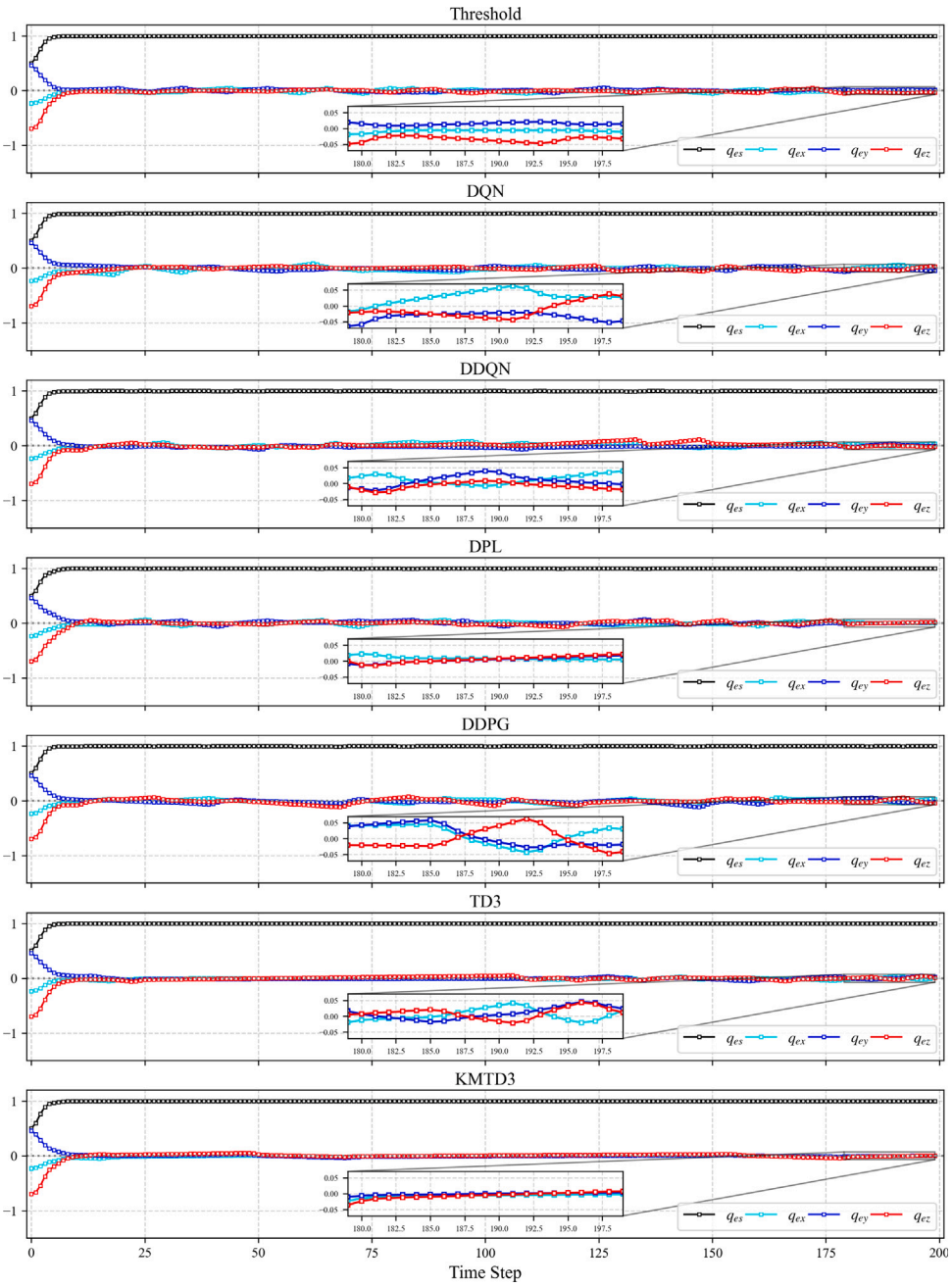


Fig. 6. Comparison of attitude error in Simulation Example 1.

Table 3
Detailed experimental results in Simulation Example 2.

Metric	Threshold	DQN	DDQN	DPL	DDPG	TD3	KMTD3
R	−6.940	−6.694	−6.682	−7.110	−6.757	−6.962	**−5.219**
	± 0.167	± 0.300	± 0.310	± 0.497	± 0.278	± 0.350	± **0.126**
J_{mpc}	5.080	5.234	5.182	5.380	5.667	5.602	**4.759**
	± 0.046	± 0.185	± 0.149	± 0.326	± 0.322	± 0.425	± **0.132**
MAE	0.052	0.053	0.052	0.055	0.063	0.060	**0.045**
	± 0.001	± 0.005	± 0.004	± 0.007	± 0.007	± 0.009	± **0.005**
MAVE	0.149	0.153	0.156	0.169	0.164	0.165	**0.074**
	± 0.013	± 0.012	± 0.015	± 0.010	± 0.011	± 0.008	± **0.005**
f_t	0.186	0.146	0.150	0.173	0.109	0.136	**0.046**
	± 0.012	± 0.016	± 0.031	± 0.048	± 0.007	± 0.021	± **0.008**
N_{tr}	37.200	29.200	30.000	34.600	21.800	27.200	**9.200**
	± 2.490	± 3.114	± 6.124	± 9.555	± 1.483	± 4.147	± **1.643**

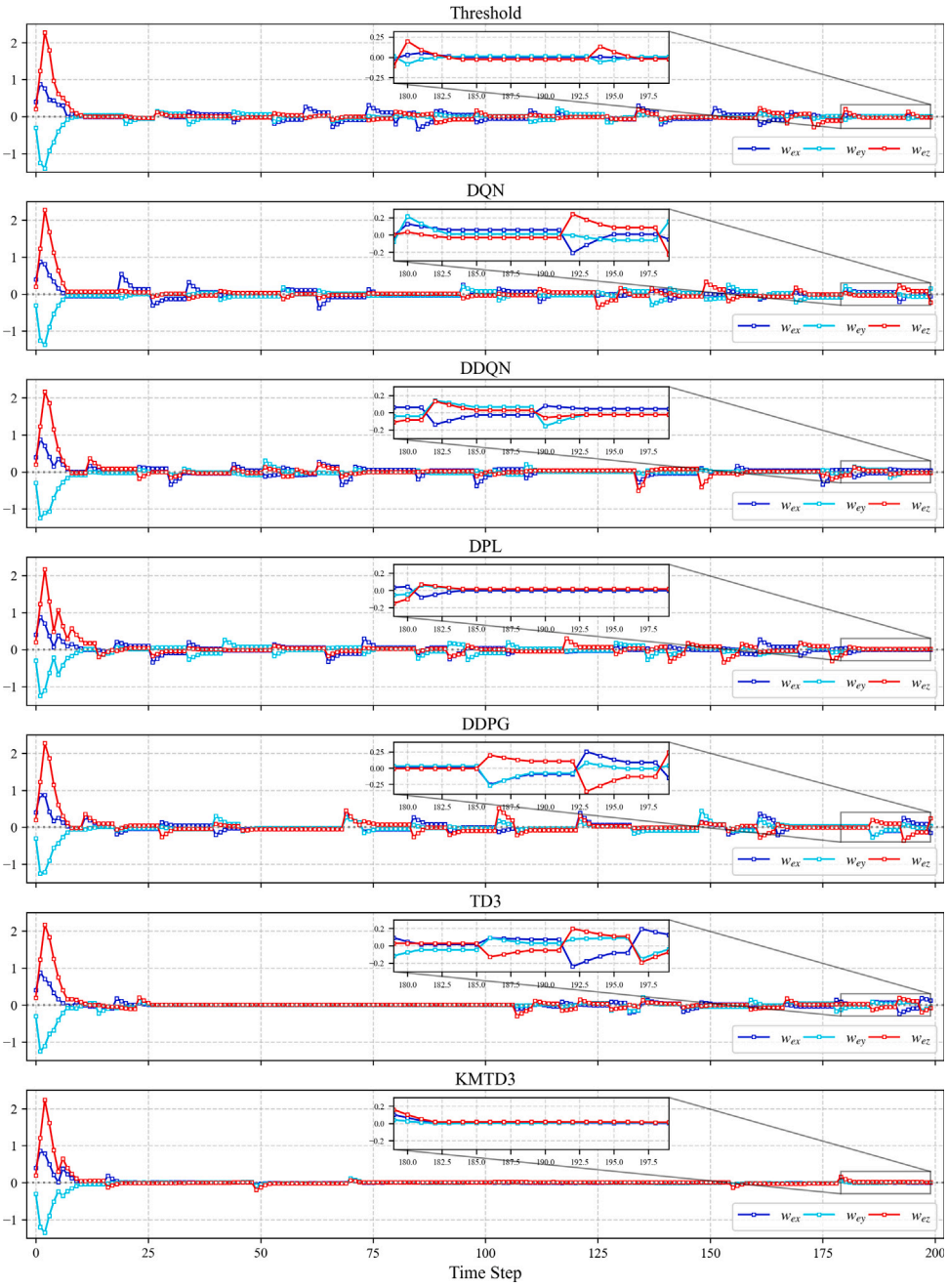


Fig. 7. Comparison of angular velocity error in Simulation Example 1.

confirming superior cumulative reward. The mean angular velocity error is 0.074, representing reductions of 50.3%, 51.6%, 52.6%, 56.2%, 54.9%, and 55.2% relative to these comparison algorithms. Moreover, the event-trigger frequency is only 0.046, corresponding to reductions of 75.3%, 68.5%, 69.3%, 73.4%, 57.8%, and 66.2%, respectively. Regarding the cumulative trigger count, KMTD3 averages only 9.2 MPC re-optimizations, fewer than half the 21.8 updates averaged by DDPG, the most trigger-efficient baseline. This result further demonstrates its ability to limit unnecessary MPC re-optimizations under challenging initial conditions. Notably, the negative-scalar error quaternion representation provides an additional test of the controller's ability to handle the double-cover property of unit quaternions. In contrast, KMTD3 achieves high-precision reorientation with a low trigger rate, demonstrating its effectiveness in this challenging simulation scenario.

Fig. 9 compares the event-trigger instants of all algorithms in a single run of Simulation Example 2. KMTD3 invokes MPC re-optimization only a few times during the initial phase, then enters an extended period without triggering. In contrast, the comparison methods continue to generate intermittent triggering events throughout the simulation. This sparse triggering pattern indicates that KMTD3 avoids unnecessary MPC re-optimizations while maintaining high attitude accuracy. Fig. 10 shows the cumulative count of MPC re-optimizations in Simulation Example 2. Most baseline curves rise throughout the maneuver, whereas TD3 exhibits a long plateau followed by a mid-stage increase. KMTD3 increases mainly in the early stage and shows only sparse increments

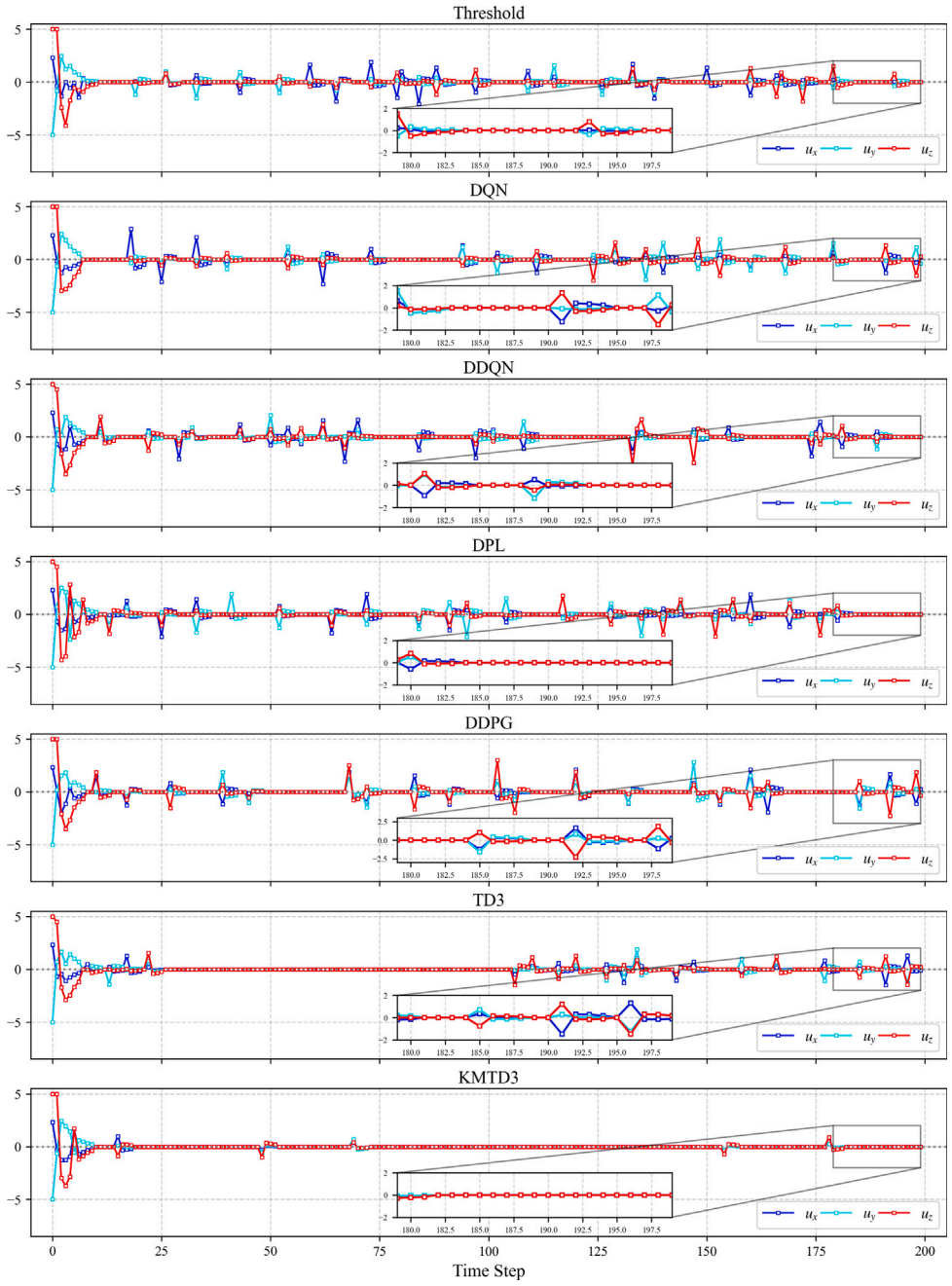


Fig. 8. Comparison of control torque in Simulation Example 1.

thereafter, indicating that it concentrates MPC re-optimizations in the most demanding phase of the large-angle maneuver. Figs. 11 and 12 show the time histories of the attitude error and angular velocity error for a representative trial, respectively. Despite the challenging initial conditions, KMTD3 rapidly drives the system toward the desired equilibrium and achieves a lower final steady-state error than the comparison methods. This highlights its ability to maintain accurate reorientation under the nonlinear dynamics associated with the large-angle initial condition. Fig. 13 displays the control torque profiles in a single run of this simulation. Unlike the pronounced high-frequency switching exhibited by other approaches, KMTD3 produces smooth torque transitions, avoiding overshoot during transients and exhibiting negligible chattering in the steady state. This low-amplitude, low-frequency torque profile may help reduce actuator activity, energy consumption, and mechanical wear. These results demonstrate that, under the large-angle initial attitude condition characterized by $q_{es} < 0$, the proposed method achieves a favorable balance between control performance and computational efficiency, indicating its potential for resource-constrained satellite attitude reorientation missions.

4.5. Ablation experiment

To further verify the effectiveness of the key components in the proposed KMTD3 framework, ablation experiments are conducted by

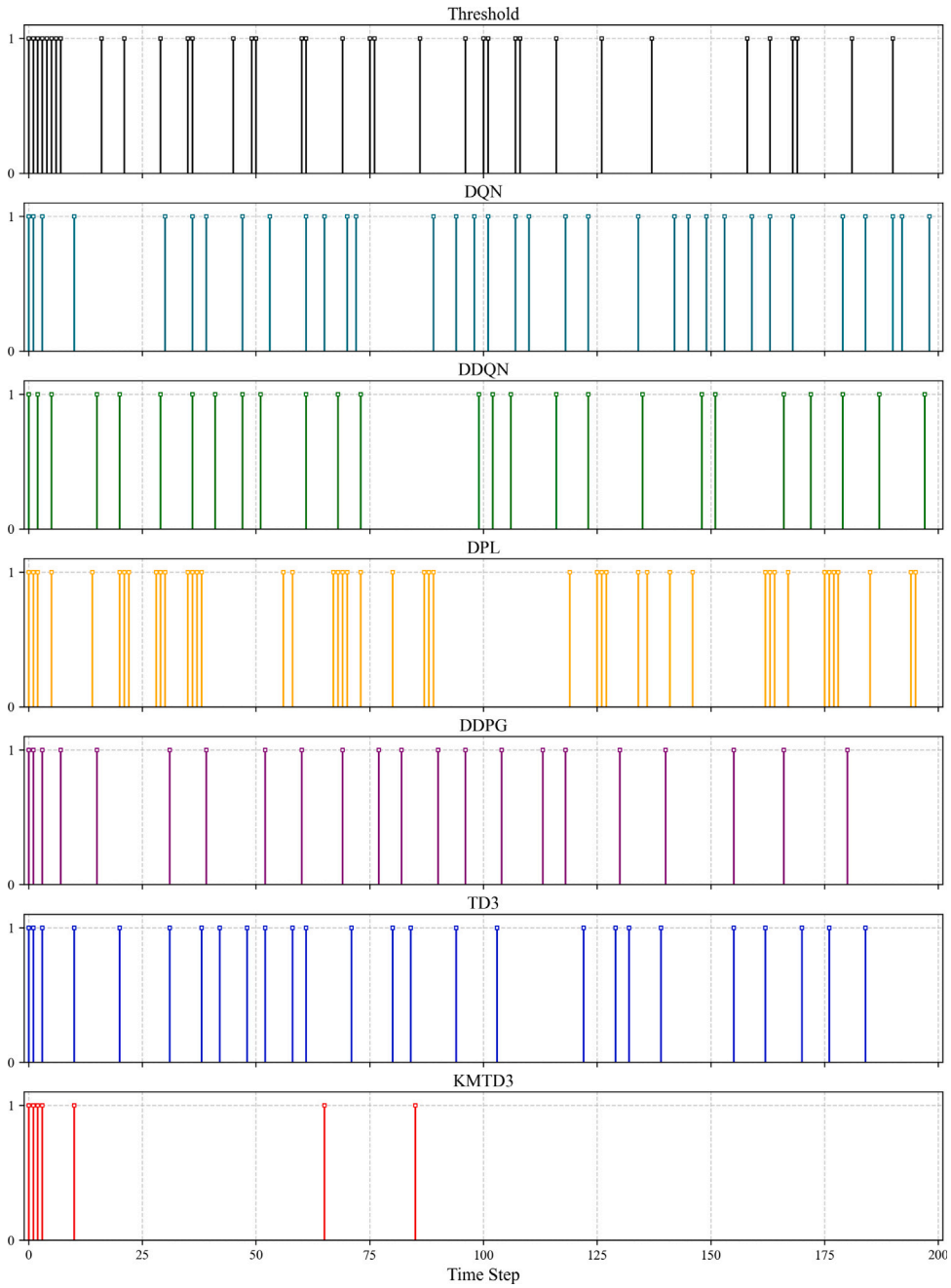


Fig. 9. Comparison of event-trigger frequency in Simulation Example 2.

comparing TD3, MTD3, KTD3, and the complete KMTD3. Here, TD3 denotes the baseline triggering policy without temporal modeling or state estimation enhancement. MTD3 incorporates the Mamba-based temporal feature extractor into TD3 to improve the assessment of historical state evolution and the validity of the cached MPC sequence. KTD3 introduces the KF into TD3 to enhance state estimation accuracy under disturbed conditions. KMTD3 integrates both the Mamba-based temporal encoder and the KF-based angular velocity estimation mechanism. The ablation results are summarized in Tables 4 and 5.

In Simulation Example 1, compared with the baseline TD3, MTD3 reduces J_{mpc}, MAE, MAVE, and f_t by 6.5%, 9.1%, 5.3%, and 3.0%, respectively. The average cumulative trigger count also decreases from 26.6 to 25.8, representing a reduction of 3.0%. This indicates that the Mamba-based temporal encoder can exploit historical state information to improve triggering decisions, although temporal modeling alone has a limited effect on disturbance suppression and trigger reduction. In contrast, KTD3 achieves more substantial improvements after introducing the Kalman filter, reducing J_{mpc}, MAE, MAVE, and f_t by 14.3%, 22.7%, 56.1%, and 55.6%, respectively, compared with TD3. The average cumulative trigger count is also reduced from 26.6 to 11.8, corresponding to a reduction of 55.6%. This more significant improvement is primarily attributed to the direct impact of Gaussian disturbances on angular velocity dynamics. Without state filtering, the triggering policy might misinterpret disturbance-induced fluctuations as actual state deviations, leading to increased angular velocity errors and unnecessary MPC re-optimizations. The KF mitigates these effects by providing more reliable state estimates for the triggering strategy and MPC initialization. In contrast, Mamba primarily enhances temporal feature extraction capabilities rather than explicitly suppressing disturbances. The complete KMTD3 further achieves the best overall performance, reducing J_{mpc}, MAE, MAVE, and f_t by 19.8%, 30.3%,

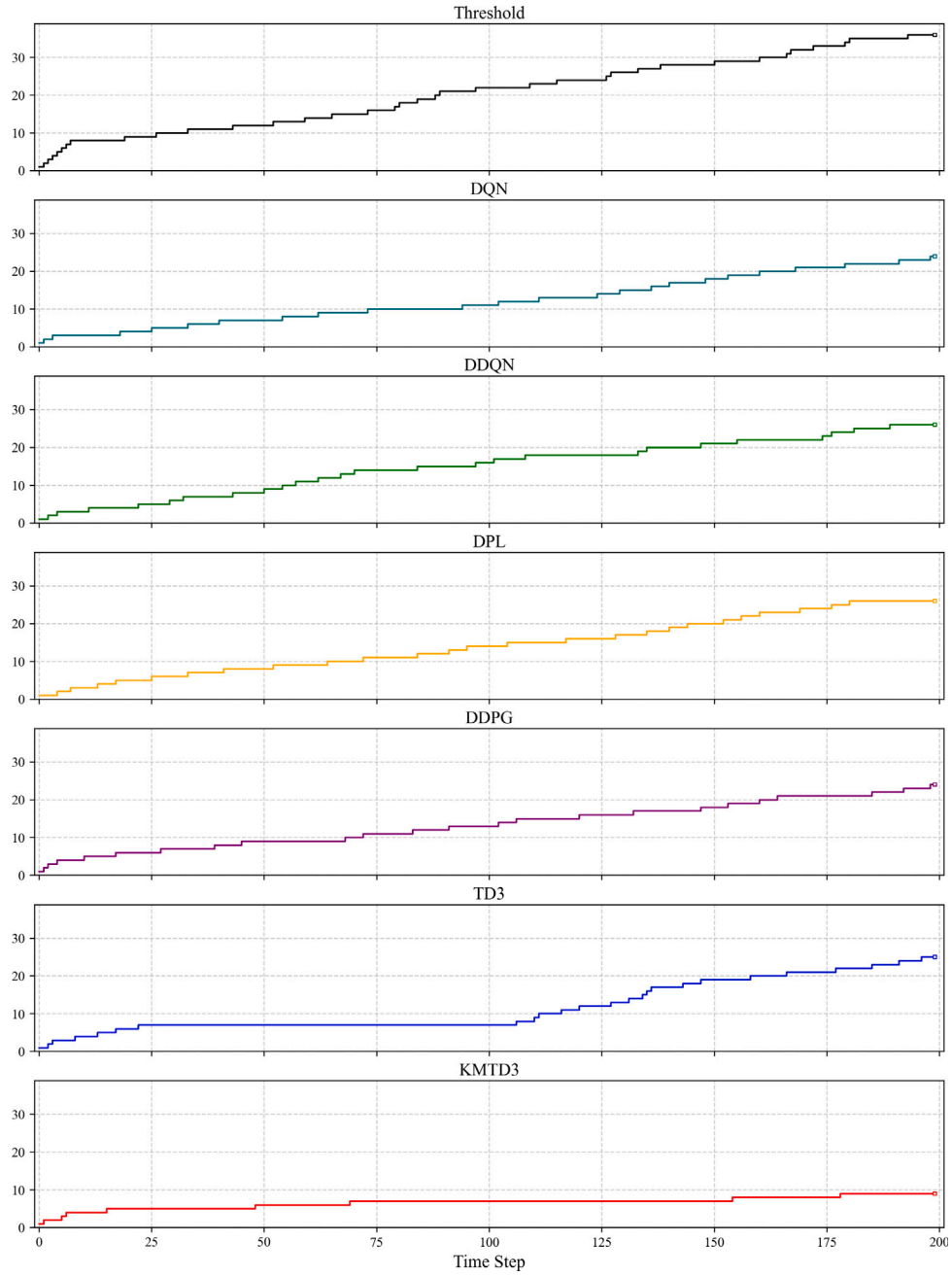


Fig. 10. Comparison of cumulative trigger counts in Simulation Example 2.

Table 4
Ablation experimental results in Simulation Example 1.

Metric	TD3	MTD3	KTD3	KMTD3
R	−7.308 ± 0.390	−6.881 ± 0.196	−5.711 ± 0.457	**−5.217 ± 0.191**
J_{mpc}	5.978 ± 0.512	5.591 ± 0.286	5.121 ± 0.488	**4.797 ± 0.241**
MAE	0.066 ± 0.009	0.060 ± 0.006	0.051 ± 0.011	**0.046 ± 0.008**
MAVE	0.171 ± 0.006	0.162 ± 0.011	0.075 ± 0.009	**0.070 ± 0.003**
f_t	0.133 ± 0.020	0.129 ± 0.023	0.059 ± 0.013	**0.042 ± 0.008**
N_{tr}	26.600 ± 3.975	25.800 ± 4.658	11.800 ± 2.588	**8.400 ± 1.673**

59.1%, and 68.4%, respectively, compared with TD3. In terms of cumulative trigger count, KMTD3 requires only 8.4 MPC re-optimizations on average, representing a reduction of 68.4% compared with the 26.6 re-optimizations required by TD3. Moreover, compared with KTD3, KMTD3 further reduces both the event-trigger frequency and the average number of MPC re-optimizations by 28.8%, confirming the complementary effect between the Mamba module and the KF. Specifically, the former improves the discrimination of closed-loop temporal evolution,

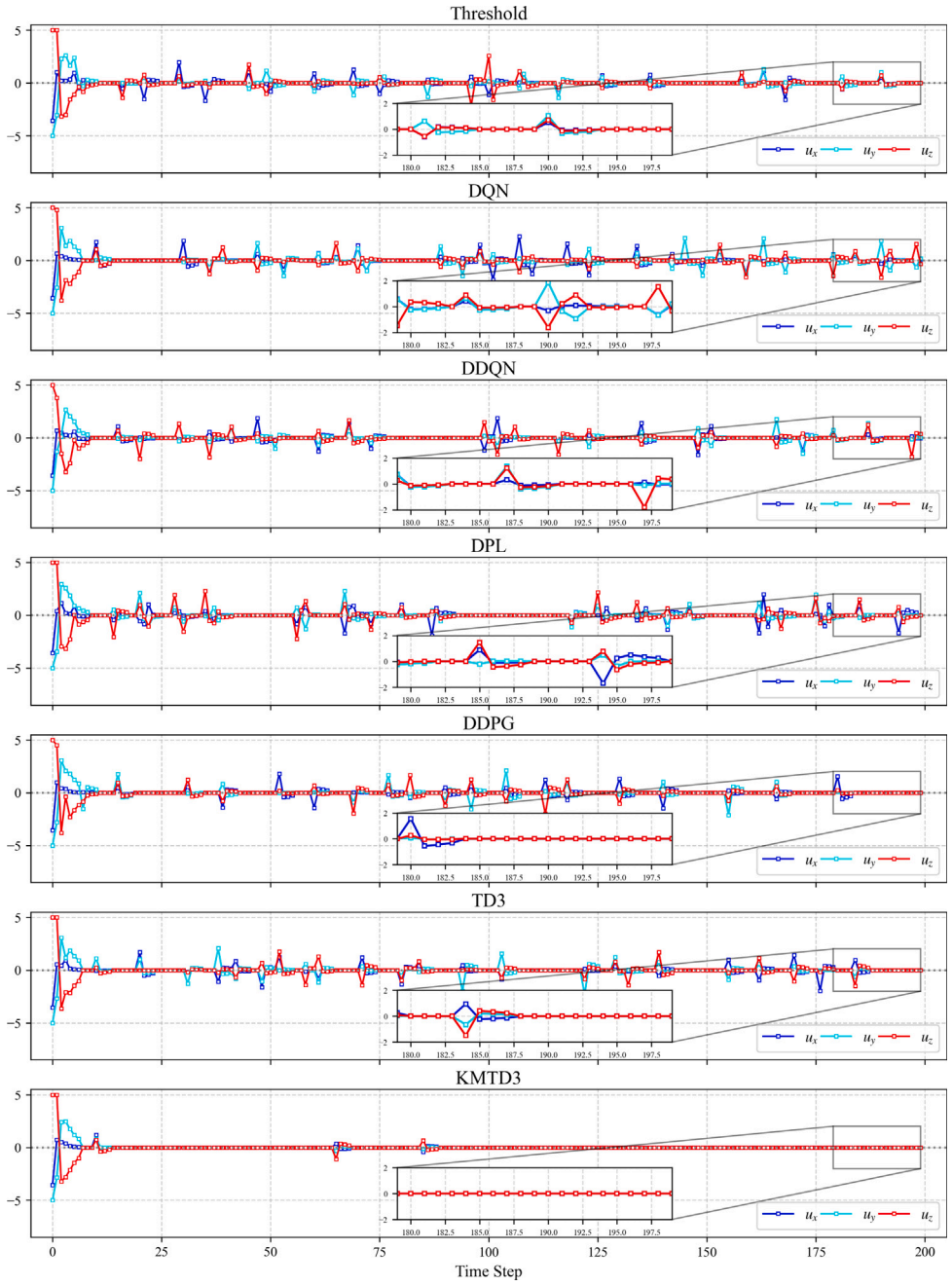


Fig. 11. Comparison of attitude error in Simulation Example 2.

Table 5
Ablation experimental results in Simulation Example 2.

Metric	TD3	MTD3	KTD3	KMTD3
R	−6.962 ± 0.350	−6.595 ± 0.259	−5.779 ± 0.484	**−5.219 ± 0.126**
J_{mpc}	5.602 ± 0.425	5.355 ± 0.199	5.319 ± 0.506	**4.759 ± 0.132**
MAE	0.060 ± 0.009	0.058 ± 0.005	0.058 ± 0.011	**0.045 ± 0.005**
MAVE	0.165 ± 0.008	0.148 ± 0.016	0.083 ± 0.008	**0.074 ± 0.005**
f_t	0.136 ± 0.021	0.124 ± 0.032	0.046 ± 0.007	**0.046 ± 0.008**
N_{tr}	27.200 ± 4.147	24.800 ± 6.496	9.200 ± 1.304	**9.200 ± 1.643**

while the latter enhances the reliability of state information. Their combination enables a more accurate judgment of whether the cached MPC control sequence remains valid.

A similar trend can be observed in Simulation Example 2. Compared with TD3, MTD3 reduces J_{mpc}, MAE, MAVE, and f_t by 4.4%, 3.3%, 10.3%, and 8.8%, respectively. The corresponding average cumulative

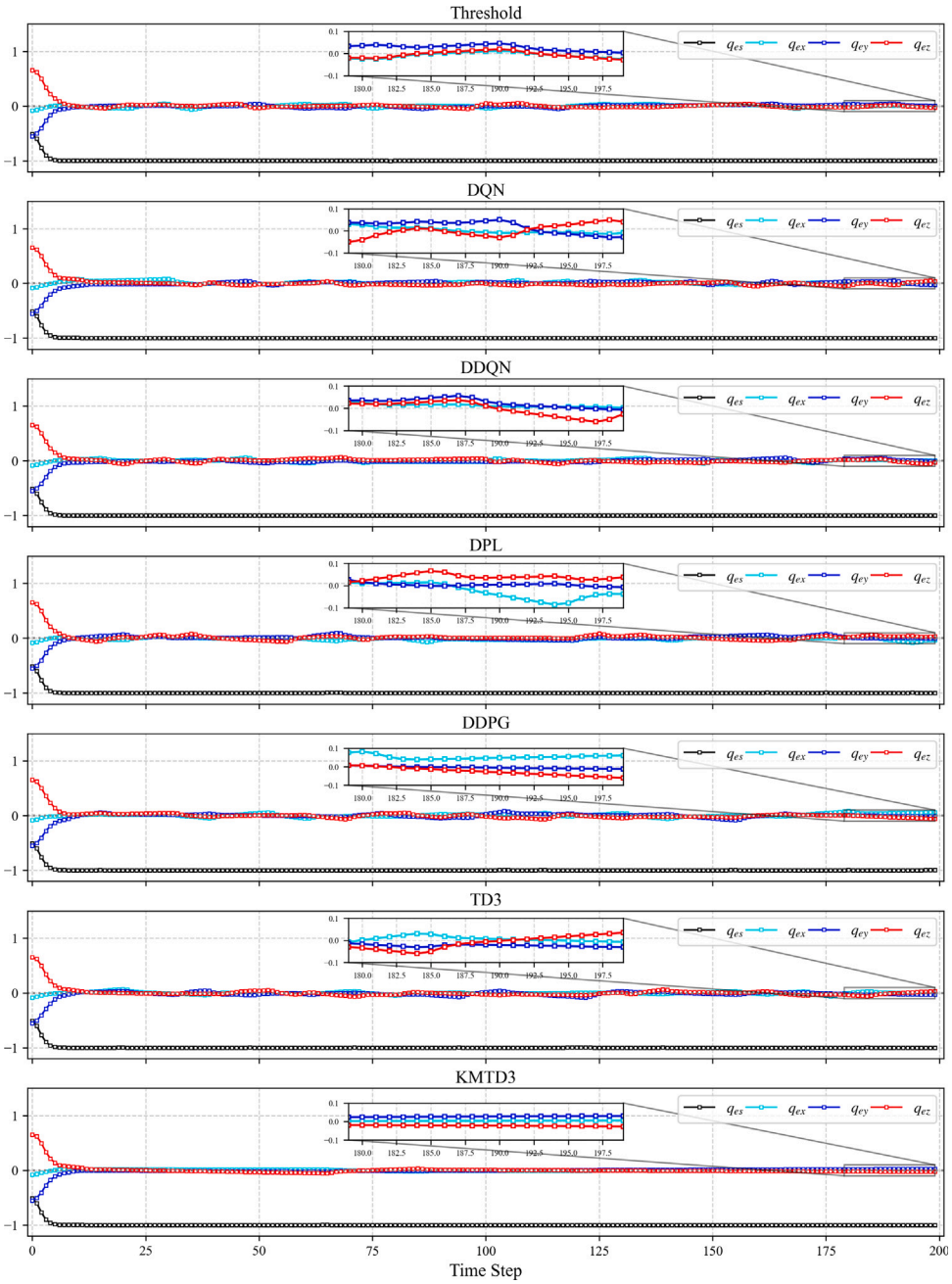


Fig. 12. Comparison of angular velocity error in Simulation Example 2.

trigger count decreases from 27.2 to 24.8, also representing a reduction of 8.8%. KTD3 reduces these metrics by 5.1%, 3.3%, 49.7%, and 66.2%, respectively. Its average cumulative trigger count is reduced from 27.2 to 9.2, corresponding to a reduction of 66.2%. The complete KMTD3 achieves the highest return of −5.219, and reduces J_{mpc}, MAE, MAVE, and f_t by 15.0%, 25.0%, 55.2%, and 66.2%, respectively, compared with TD3. The reduction in triggering activity is also reflected in N_{tr}, which decreases from 27.2 for TD3 to 9.2 for KMTD3. Notably, in this large-angle reorientation scenario, KMTD3 and KTD3 achieve the same event-trigger frequency of 0.046. Both methods also yield the same average cumulative trigger count of 9.2, showing that the performance gains of KMTD3 are obtained without additional MPC re-optimizations. However, compared with KTD3, KMTD3 further reduces J_{mpc}, MAE, and MAVE by 10.5%, 22.4%, and 10.8%, respectively. This indicates that even under the same triggering frequency and MPC update count, the Mamba-based temporal modeling module can still further improve control accuracy and cumulative performance.

In summary, the ablation experiments demonstrate that the Kalman filter mainly improves the reliability of angular velocity estimation under Gaussian measurement disturbances, thereby reducing angular velocity errors and unnecessary MPC re-optimizations. The Mamba-based temporal encoder enhances the policy's ability to understand historical closed-loop evolution, allowing the cached MPC control sequence to be reused more appropriately. By combining these two mechanisms, KMTD3 achieves a better balance among control accuracy, trigger sparsity, and cumulative return, verifying the effectiveness of the proposed Gaussian-disturbance-aware temporal triggering framework.

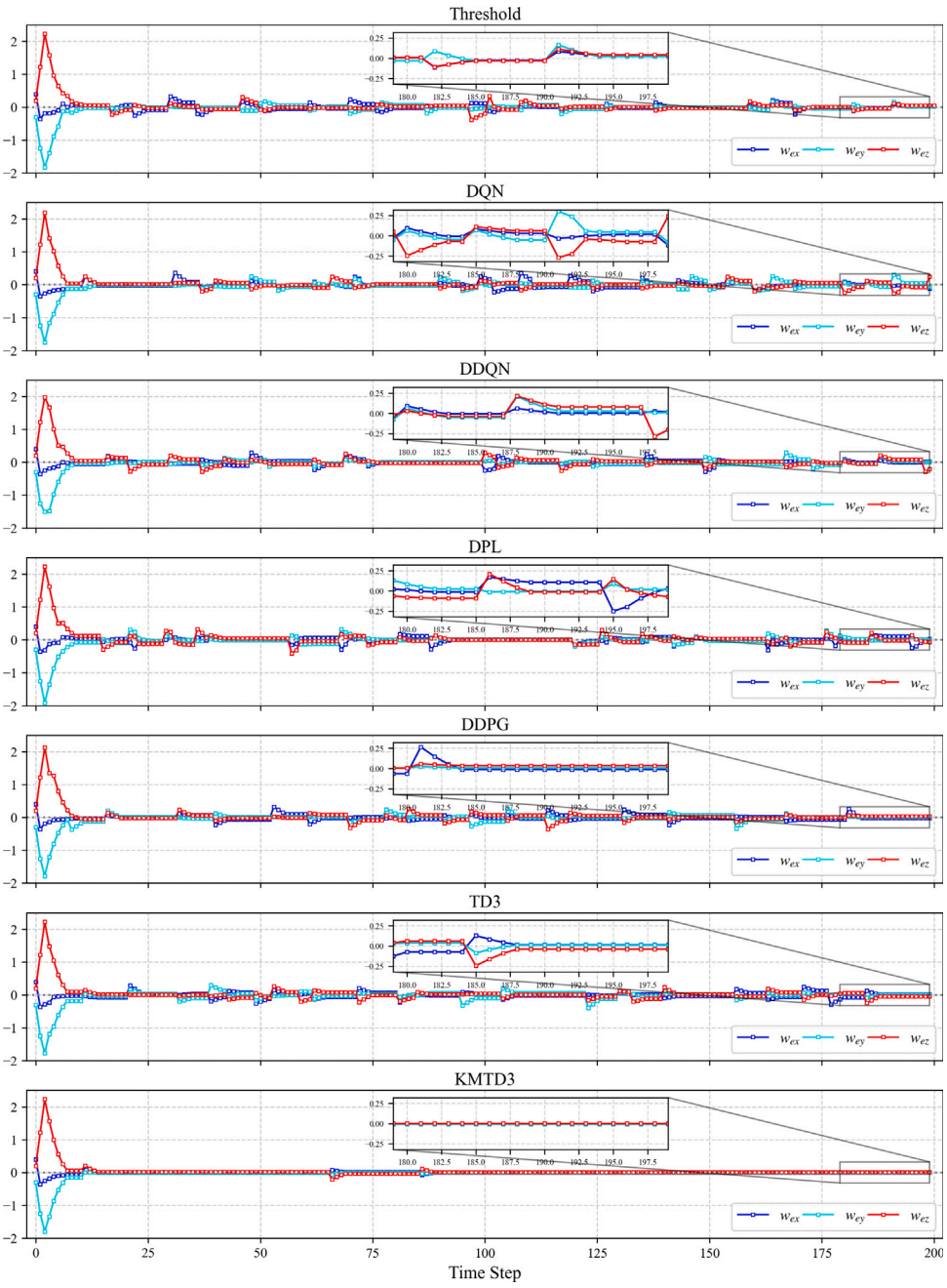


Fig. 13. Comparison of control torque in Simulation Example 2.

4.6. Parameter λ comparison experiment

To further investigate the influence of the triggering penalty on the trade-off between MPC re-optimization frequency and control performance, a sensitivity analysis of the penalty weight λ is conducted. In the reward function, λ directly penalizes MPC re-optimizations and therefore determines how strongly the policy is encouraged to reduce unnecessary MPC re-optimizations. A smaller λ generally favors reorientation accuracy by allowing more frequent MPC re-optimizations, whereas a larger λ encourages sparser triggering at the possible cost of degraded control performance. Four representative values, i.e., $\lambda = 0$, $\lambda = 0.02$, $\lambda = 0.05$, and $\lambda = 0.07$, are tested in both simulation examples, and the corresponding results are summarized in Tables 6 and 7.

Table 6 summarizes the sensitivity analysis results of KMTD3 in Simulation Example 1. When $\lambda = 0$, no penalty is imposed on MPC re-optimizations. Consequently, the controller tends to trigger MPC re-optimization at almost every time step, resulting in an event-trigger frequency of 0.999. Correspondingly, the average cumulative trigger count reaches 199.8 over the 200-step simulation, confirming that MPC is re-optimized at almost every time step. This setting achieves the lowest total cost of 4.313, the smallest mean attitude error of 0.021, and the smallest mean angular velocity error of 0.058. However, such nearly continuous triggering effectively converts the event-triggered mechanism into a time-triggered MPC scheme, thereby eliminating the computational advantage of ET-MPC.

When λ is increased to 0.02, the event-trigger frequency is sharply reduced from 0.999 to 0.062, indicating that the triggering penalty effectively suppresses unnecessary MPC re-optimizations. The corresponding average cumulative trigger count decreases to 12.4. Meanwhile, the total cost, mean attitude error, and mean angular velocity error increase to 4.906, 0.048, and 0.083, respectively. This result shows

Table 6
Comparison experimental results on KMTD3 in Simulation Example 1.

Metric	$\lambda = 0$	$\lambda = 0.02$	$\lambda = 0.05$	$\lambda = 0.07$
R	−4.313 ± 0.009	−5.154 ± 0.181	−5.217 ± 0.191	−5.916 ± 0.509
J_{mpc}	4.313 ± 0.009	4.906 ± 0.171	4.797 ± 0.241	5.300 ± 0.525
MAE	0.021 ± 0.001	0.048 ± 0.004	0.046 ± 0.008	0.056 ± 0.011
MAVE	0.058 ± 0.001	0.083 ± 0.005	0.070 ± 0.003	0.092 ± 0.011
f_t	0.999 ± 0.002	0.062 ± 0.004	0.042 ± 0.008	0.044 ± 0.007
N_{tr}	199.800 ± 0.447	12.400 ± 0.894	8.400 ± 1.673	8.800 ± 1.304

Table 7
Comparison experimental results on KMTD3 in Simulation Example 2.

Metric	$\lambda = 0$	$\lambda = 0.02$	$\lambda = 0.05$	$\lambda = 0.07$
R	−4.290 ± 0.007	−5.137 ± 0.438	−5.219 ± 0.126	−6.038 ± 0.586
J_{mpc}	4.290 ± 0.007	4.893 ± 0.453	4.759 ± 0.132	5.296 ± 0.397
MAE	0.022 ± 0.001	0.047 ± 0.010	0.045 ± 0.005	0.056 ± 0.008
MAVE	0.057 ± 0.001	0.078 ± 0.010	0.074 ± 0.005	0.090 ± 0.010
f_t	0.999 ± 0.002	0.061 ± 0.014	0.046 ± 0.008	0.053 ± 0.016
N_{tr}	199.800 ± 0.447	12.200 ± 2.775	9.200 ± 1.643	10.600 ± 3.130

that a moderate triggering penalty can substantially reduce computational burden while maintaining acceptable reorientation performance. Further increasing λ to 0.05 reduces the event-trigger frequency to 0.042. The average cumulative trigger count is further reduced to 8.4, meaning that only a small number of MPC re-optimizations are required throughout the maneuver. More importantly, compared with $\lambda = 0.02$, this setting also improves the total cost from 4.906 to 4.797, the mean attitude error from 0.048 to 0.046, and the mean angular velocity error from 0.083 to 0.070. Therefore, $\lambda = 0.05$ provides a more favorable balance between triggering sparsity and control accuracy in Simulation Example 1.

By contrast, when $\lambda = 0.07$, the event-trigger frequency remains low at 0.044, corresponding to an average cumulative trigger count of 8.8, but the total cost, mean attitude error, and mean angular velocity error increase to 5.300, 0.056, and 0.092, respectively. This indicates that an excessively large triggering penalty may cause the policy to trigger MPC re-optimization too infrequently, thereby delaying necessary feedback corrections and degrading steady-state performance. These results demonstrate that λ should not be selected solely to minimize the trigger frequency; instead, it must be chosen by jointly considering reorientation accuracy, angular velocity suppression, and the sparsity of MPC re-optimizations.

Table 7 presents the corresponding sensitivity analysis results in Simulation Example 2. A similar trend can be observed under the more challenging large-angle reorientation condition. When $\lambda = 0$, KMTD3 again achieves the best tracking metrics, with a total cost of 4.290, a mean attitude error of 0.022, and a mean angular velocity error of 0.057. Nevertheless, the event-trigger frequency reaches 0.999, with a corresponding cumulative trigger count of 199.8, indicating that MPC is re-optimized at almost every time step. Although this setting provides high control accuracy, it contradicts the objective of reducing onboard computational burden.

For $\lambda = 0.02$, the event-trigger frequency decreases dramatically to 0.061, and the cumulative trigger count decreases from 199.800 to 12.200, while the total cost, mean attitude error, and mean angular velocity error become 4.893, 0.047, and 0.078, respectively. This confirms that the triggering penalty remains effective in reducing unnecessary MPC re-optimizations even under the quaternion condition $q_{es} < 0$. When $\lambda = 0.05$, the event-trigger frequency is further reduced to 0.046. The corresponding cumulative trigger count is further reduced to 9.200. At the same time, this setting achieves a lower total cost of 4.759, a smaller mean attitude error of 0.045, and a smaller mean angular velocity error of 0.074 compared with $\lambda = 0.02$. These results indicate that $\lambda = 0.05$ yields the best overall trade-off among the nonzero penalty settings in Simulation Example 2.

When $\lambda = 0.07$, the event-trigger frequency remains low at 0.053, with a corresponding cumulative trigger count of 10.600, but the total cost increases to 5.296, while the mean attitude error and mean angular velocity error increase to 0.056 and 0.090, respectively. The slight increases in both the triggering frequency and cumulative trigger count compared with $\lambda = 0.05$ also suggest that, due to the nonlinear closed-loop dynamics and stochastic policy learning process, increasing the triggering penalty does not necessarily lead to a strictly monotonic reduction in trigger frequency. Instead, an overly large penalty may cause suboptimal triggering timing, which deteriorates state regulation and eventually requires additional corrective MPC re-optimizations.

Overall, the sensitivity analysis shows that the triggering penalty weight λ has a direct influence on the balance between control accuracy and the sparsity of MPC re-optimizations. When $\lambda = 0$, the controller tends to invoke MPC at almost every time step, resulting in the best tracking accuracy but losing the computational advantage of ET-MPC. As λ increases from zero to a moderate value, both the event-trigger frequency and the cumulative trigger count are significantly reduced, whereas an excessively large penalty may degrade the total cost and steady-state accuracy. Among the tested settings, $\lambda = 0.05$ provides a favorable trade-off between reorientation accuracy, angular velocity suppression, and triggering efficiency in both simulation examples. These results further confirm that the proposed KMTD3 framework can achieve a favorable balance between control performance and MPC re-optimization frequency when the triggering penalty is appropriately selected.

4.7. Computational cost analysis

To determine whether the reduction in MPC triggering frequency translates into an actual reduction in online computational cost, wall-clock timing experiments were conducted for both simulation examples. For each example, five KMTD3 policy checkpoints obtained from training runs with independent random seeds were evaluated, and each timing test was repeated three times to reduce measurement variability. Warm-up calls were excluded to avoid including one-time initialization overhead in the reported online execution times. The per-step KMTD3 inference overhead, MPC re-optimization time, episode-level controller cost, and peak triggered-step load were measured and compared with those of the periodic MPC baseline. The execution times of the individual components for the two simulation examples are reported in Table 8, while the accumulated episode-level online costs and measured peak controller-step loads are presented in Tables 9 and 10. These results help identify the sources of computational overhead and assess whether the reduced MPC re-optimization frequency yields a net online computational benefit.

As shown in Table 8, the complete KMTD3 agent required an average of 1.626 ms per step in Simulation Example 1 and 1.678 ms per step in Simulation Example 2. By comparison, one complete MPC

Table 8
Measured wall-clock time of each component in Simulation Examples 1 and 2.

Component	Simulation Example 1	Simulation Example 2
KF update (ms)	0.051 ± 0.002	0.051 ± 0.004
Mamba encoder (ms)	1.097 ± 0.064	1.073 ± 0.029
TD3 actor head (ms)	0.140 ± 0.009	0.127 ± 0.005
Complete KMTD3 agent (ms)	1.626 ± 0.105	1.678 ± 0.045
MPC problem construction (ms)	45.997 ± 0.922	45.274 ± 1.802
IPOPT solution time (ms)	5.747 ± 0.378	5.524 ± 0.254
Complete MPC re-optimization (ms)	52.281 ± 1.297	51.314 ± 1.912

Table 9
Episode-level online computational costs for Simulation Example 1.

Metric	KMTD3	Periodic MPC
MPC re-optimizations per episode	8.400 ± 1.673	200.000 ± 0.000
Trigger frequency	0.042 ± 0.008	1.000 ± 0.000
Controller cost per episode (ms)	767.775 ± 94.494	10246.980 ± 98.393
Controller cost per step (ms)	3.839 ± 0.473	51.235 ± 0.492
Computational saving (%)	92.503 ± 0.965	–
Maximum controller cost per step (ms)	84.603	95.784

Table 10
Episode-level online computational costs for Simulation Example 2.

Metric	KMTD3	Periodic MPC
MPC re-optimizations per episode	9.200 ± 1.643	200.000 ± 0.000
Trigger frequency	0.046 ± 0.008	1.000 ± 0.000
Controller cost per episode (ms)	810.950 ± 71.791	10006.341 ± 54.730
Controller cost per step (ms)	4.055 ± 0.359	50.032 ± 0.274
Computational saving (%)	91.894 ± 0.734	–
Maximum controller cost per step (ms)	112.482	98.178

re-optimization required 52.281 ms and 51.314 ms, respectively. The KMTD3 inference overhead per step therefore accounted for only approximately 3.1% and 3.3% of the corresponding MPC re-optimization cost.

Although the prediction horizon was short, the measured MPC cost was not limited to the IPOPT numerical solution. MPC problem construction required approximately 46.000 ms in Simulation Example 1 and 45.274 ms in Simulation Example 2, accounting for approximately 88.0% and 88.2% of the complete re-optimization time, respectively. In contrast, the KF update, Mamba encoder, and TD3 actor head required substantially less execution time. These results indicate that the additional inference overhead introduced by KMTD3 does not offset the computational benefit of avoiding unnecessary MPC re-optimizations.

As reported in Table 9, KMTD3 reduced the average number of MPC re-optimizations from 200.000 to 8.400 per episode in Simulation Example 1, corresponding to an average trigger frequency of 0.042. Consequently, the episode-level controller cost decreased from 10246.980 ms to 767.775 ms, yielding a computational saving of 92.5%. Similarly, Table 10 shows that KMTD3 performed an average of 9.200 MPC re-optimizations per episode in Simulation Example 2, with a trigger frequency of 0.046. The episode-level controller cost was reduced from 10006.341 ms for periodic MPC to 810.950 ms for KMTD3, corresponding to a computational saving of 91.9%.

The average controller cost per step decreased from 51.235 ms to 3.839 ms in Simulation Example 1 and from 50.032 ms to 4.055 ms in Simulation Example 2. The maximum measured KMTD3 controller costs per step were 84.603 ms and 112.482 ms, respectively. Both values remained below the 200-ms sampling interval, and no deadline misses were observed during the timing experiments. Nevertheless, these maximum values are platform-specific measurements and should not be interpreted as hardware-independent guarantees on worst-case execution time.

Overall, after all per-step inference and event-dependent MPC costs were included, KMTD3 reduced the measured episode-level online computational cost by more than 91.0% in both simulation examples, while the observed peak controller-step cost remained within the sampling period.

Nevertheless, the workstation-based timing results do not establish real-time implementability on flight-representative embedded processors. During deployment, the online controller retains the KF, the Mamba-based actor, and the MPC optimizer, whereas the critic networks and other training-related components are removed. The deployed actor contains 115,969 parameters, equivalent to approximately 0.116 million parameters and a theoretical FP32 parameter storage requirement of approximately 0.442 MiB. The KF consists of low-dimensional matrix operations and incurred a measured overhead of only approximately 0.051 ms per step in both simulation examples, as reported in Table 8. However, the computational capabilities and software environments of resource-constrained flight processors differ substantially from those of the desktop CPU/GPU platform used in this study. These differences highlight the importance of embedded inference efficiency for practical deployment. Therefore, hardware-in-the-loop evaluation on flight-representative processors, together with quantization, model compression, and embedded inference optimization, will be considered in future work.

5. Conclusion

This paper proposes KMTD3, a novel data-driven intelligent framework for satellite attitude reorientation. To the best of our knowledge, it is the first framework to integrate DRL with ET-MPC for satellite attitude control. By leveraging a learned state-dependent policy, KMTD3 generates triggering decisions online to balance control performance and MPC re-optimization frequency. The integration of the Mamba-based SSM enhances temporal modeling of recent state histories, thereby helping reduce unnecessary MPC re-optimizations. Additionally, the reduced-order KF provides more reliable angular velocity estimates under Gaussian measurement noise, thereby improving robustness to disturbances in the angular velocity measurements. Simulation results demonstrate that KMTD3 achieves improved control accuracy and triggering efficiency under Gaussian disturbances in the angular velocity measurements compared with the considered benchmark methods. The wall-clock timing experiments further show that

KMTD3 substantially reduces the measured online computational cost relative to periodic MPC on the tested computing platform. Overall, the proposed framework achieves a favorable balance between control performance and computational efficiency. The present study employs a reduced-order KF to estimate the three-dimensional angular velocity under Gaussian measurement noise. Further studies will investigate MEKF, UKF, and CKF methods for full nonlinear attitude estimation involving noisy quaternion measurements and additional sensor states, together with more realistic non-Gaussian and time-varying disturbance models. Although the measured execution times remained within the sampling period on the tested workstation, real-time implementability on resource-constrained flight hardware has not yet been established. Future work will therefore evaluate KMTD3 on flight-representative embedded processors through hardware-in-the-loop experiments and investigate quantization, model compression, and embedded inference optimization. It is also important to note that the present study is limited to single-satellite scenarios and does not address key challenges in multi-satellite coordination, such as dynamic coupling, communication constraints, and distributed task allocation. The proposed framework will subsequently be extended to multi-satellite constellations by developing a collaborative attitude control architecture that supports distributed decision-making and inter-satellite state sharing. This extension is expected to support more autonomous and resource-efficient satellite constellations.

CRediT authorship contribution statement

Liushuai Zheng: Writing – original draft, Visualization, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Yadong Chen:** Writing – review & editing, Supervision. **Zixuan Zhou:** Writing – review & editing, Supervision. **Peng Cheng:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was funded by the Sichuan Provincial Science and Technology Program (Grant No. 2024ZDZX0046), and the Open Project Program of National Key Laboratory of Space-Born Intelligent Information Processing (Grant No. YJ-03-25-04).

References

[1] F. Zhang, G. Duan, Coupled dynamics and integrated control for position and attitude motions of spacecraft: A survey, IEEE/CAA J. Autom. Sin. 10 (12) (2023) 2187–2208.
[2] M. Yaqoob, A. Lashab, J.C. Vasquez, J.M. Guerrero, M.E. Orchard, A.D. Bintoudi, A comprehensive review on small satellite microgrids, IEEE Trans. Power Electron. 37 (10) (2022) 12741–12762.
[3] M. Khodaverdian, M. Malekzadeh, Fault-tolerant model predictive sliding mode control with fixed-time attitude stabilization and vibration suppression of flexible spacecraft, Aerosp. Sci. Technol. 139 (2023) 108381.
[4] K. Jin, G. Huang, R. Lei, C. Wei, Y. Wang, Multi-constrained predictive optimal control for spacecraft attitude stabilization and tracking with performance guarantees, Aerosp. Sci. Technol. 155 (2024) 109599.
[5] P. Iannelli, F. Angeletti, P. Gasbarri, A model predictive control for attitude stabilization and spin control of a spacecraft with a flexible rotating payload, Acta Astronaut. 199 (2022) 401–411.
[6] M. Bujarbaruah, U. Rosolia, Y.R. Stürz, F. Borrelli, A simple robust MPC for linear systems with parametric and additive uncertainty, in: 2021 American Control Conference, ACC, IEEE, 2021, pp. 2108–2113.
[7] Y. Zhou, Y. Hu, K.-V. Ling, F. Ding, Hybrid two-stage identification-based nonlinear mpc strategy for satellite attitude control, IEEE Trans. Aerosp. Electron. Syst. (2025).
[8] M. Mammarella, E. Capello, H. Park, G. Guglieri, M. Romano, Tube-based robust model predictive control for spacecraft proximity operations in the presence of persistent disturbance, Aerosp. Sci. Technol. 77 (2018) 585–594.
[9] X. Wang, Stochastic optimal control of spacecraft attitude stabilization, in: 2024 IEEE International Conference on Robotics and Biomimetics, ROBIO, IEEE, 2024, pp. 359–364.
[10] J. Yao, Q. Hu, J. Zheng, Nonlinear optimal attitude control of spacecraft using novel state-dependent coefficient parameterizations, Aerosp. Sci. Technol. 112 (2021) 106586.
[11] L. Ding, Q.-L. Han, X. Ge, X.-M. Zhang, An overview of recent advances in event-triggered consensus of multiagent systems, IEEE Trans. Cybern. 48 (4) (2017) 1110–1123.
[12] R. Zheng, R. Hao, H. Yu, D. Shi, Event-triggered lifted robust MPC stabilization control for space telescope on dual-rate actuated rigid spacecraft systems, Aerosp. Sci. Technol. 155 (2024) 109553.
[13] Q. Liu, M. Liu, Y. Shi, J. Yu, Event-triggered adaptive attitude control for flexible spacecraft with actuator nonlinearity, Aerosp. Sci. Technol. 106 (2020) 106111.
[14] N. He, X. Feng, Z. Xu, D. Tian, F. Cheng, H. Li, Event-triggered distributed MPC for cyber-physical systems: An adaptive dual-horizon mechanism, IEEE Trans. Autom. Sci. Eng. (2025).
[15] L. Wang, G. Wen, J. Zhou, H. Zhu, Event-triggered finite-time attitude control for multiple rigid spacecraft with user-defined attitude constraints, IEEE Trans. Aerosp. Electron. Syst. (2025).
[16] W. Liu, Y. Geng, B. Wu, D. Wang, Neural-network-based adaptive event-triggered control for spacecraft attitude tracking, IEEE Trans. Neural Netw. Learn. Syst. 31 (10) (2019) 4015–4024.
[17] Y. Shi, Q. Hu, D. Li, M. Lv, Adaptive optimal tracking control for spacecraft formation flying with event-triggered input, IEEE Trans. Ind. Informatics 19 (5) (2022) 6418–6428.
[18] F. Zhang, B. Huang, P. Huang, Adaptive predefined-time dual channel event-triggered deployment control of triangle space tethered system, IEEE Trans. Aerosp. Electron. Syst. (2024).
[19] Z. Zhou, Y. Chen, P. Cheng, H. Cheng, Deep learning-enhanced koopman operator linearization with LQR control for satellite attitude regulation, Adv. Space Res. (2025).
[20] Y. Chen, Z. Zhou, C. Wan, P. Cheng, Deep learning-enhanced koopman-operator-based self-triggered distributionally robust optimization SMPC strategy for satellite attitude control, Knowl.-Based Syst. (2025) 114752.
[21] S. Sharma, S. D'Amico, Neural network-based pose estimation for noncooperative spacecraft rendezvous, IEEE Trans. Aerosp. Electron. Syst. 56 (6) (2020) 4638–4658.
[22] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, M. Riedmiller, Deterministic policy gradient algorithms, in: International Conference on Machine Learning, Pmlr, 2014, pp. 387–395.
[23] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, 2015, arXiv preprint arXiv:1509.02971.
[24] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 30, (1) 2016.
[25] S. Fujimoto, H. Hoof, D. Meger, Addressing function approximation error in actor-critic methods, in: International Conference on Machine Learning, PMLR, 2018, pp. 1587–1596.
[26] L. Pan, Q. Cai, L. Huang, Softmax deep double deterministic policy gradients, Adv. Neural Inf. Process. Syst. 33 (2020) 11767–11777.
[27] B.R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A.A. Al Sallab, S. Yogamani, P. Pérez, Deep reinforcement learning for autonomous driving: A survey, IEEE Trans. Intell. Transp. Syst. 23 (6) (2021) 4909–4926.
[28] F. Dang, D. Chen, J. Chen, Z. Li, Event-triggered model predictive control with deep reinforcement learning for autonomous driving, IEEE Trans. Intell. Veh. 9 (1) (2023) 459–468.
[29] J. Lu, L. Li, F.-Y. Wang, Event-triggered parallel control using deep reinforcement learning with application to comfortable autonomous driving, IEEE Trans. Intell. Veh. 9 (3) (2024) 4470–4479.
[30] C. Tang, B. Abbatematteo, J. Hu, R. Chandra, R. Martín-Martín, P. Stone, Deep reinforcement learning for robotics: A survey of real-world successes, Annu. Rev. Control. Robot. Auton. Syst. 8 (1) (2025) 153–188.
[31] M. Wang, P. Zhang, G. Zhang, K. Sun, J. Zhang, M. Jin, A resilient scheduling framework for multi-robot multi-station welding flow shop scheduling against robot failures, Robot. Comput.-Integr. Manuf. 91 (2025) 102835.
[32] Z. Li, C. Zeng, Z. Deng, Q. Xu, B. He, J. Zhang, Learning variable impedance control for robotic massage with deep reinforcement learning: A novel learning framework, IEEE Syst. Man Cybern. Mag. 10 (1) (2024) 17–27.
[33] G. Zhou, W. Tian, R. Buyya, R. Xue, L. Song, Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions, Artif. Intell. Rev. 57 (5) (2024) 124.
[34] K. Cui, J. Song, L. Zhang, Y. Tao, W. Liu, D. Shi, Event-triggered deep reinforcement learning for dynamic task scheduling in multisatellite resource allocation, IEEE Trans. Aerosp. Electron. Syst. 59 (4) (2022) 3766–3777.

[35] G. Li, X. Li, J. Li, J. Chen, X. Shen, PTMB: An online satellite task scheduling framework based on pre-trained Markov decision process for multi-task scenario, Knowl.-Based Syst. 284 (2024) 111339.
[36] D. Zhou, G. Sun, W. Lei, L. Wu, Space noncooperative object active tracking with deep reinforcement learning, IEEE Trans. Aerosp. Electron. Syst. 58 (6) (2022) 4902–4916.
[37] Y. Hu, D. Zhou, W. Yao, X. Shao, G. Sun, Deep reinforcement learning-based trajectory planning with continuous pose representation for 6-dof free-floating space robot, Aerosp. Sci. Technol. 166 (2025) 110540.
[38] Y. Hu, D. Zhou, K. Ouyang, Z. Yu, J. Lv, X. Shao, Prior policy guided dual-agent coordinated manipulation planning of spacecraft-manipulator system, Acta Astronaut. (2026).
[39] R. Su, F. Wu, J. Zhao, Deep reinforcement learning method based on DDPG with simulated annealing for satellite attitude control system, in: 2019 Chinese Automation Congress, CAC, IEEE, 2019, pp. 390–395.
[40] D. Gao, H. Zhang, C. Li, X. Gao, Satellite attitude control with deep reinforcement learning, in: 2020 Chinese Automation Congress, CAC, IEEE, 2020, pp. 4095–4101.
[41] H. Li, Q. Zong, X. Zhang, Anti-collision trajectory planning for satellite formation reconstruction based on deep reinforcement learning, in: 2022 41st Chinese Control Conference, CCC, IEEE, 2022, pp. 4672–4677.
[42] A. Gu, T. Dao, Mamba: Linear-time sequence modeling with selective state spaces, in: First Conference on Language Modeling, 2024.
[43] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).
[44] Q. Lv, X. Deng, G. Chen, M.Y. Wang, L. Nie, Decision mamba: A multi-grained state space model with self-evolution regularization for offline rl, Adv. Neural Inf. Process. Syst. 37 (2024) 22827–22849.
[45] Y. Lei, J. Xue, L. Zhao, C. Yue, K. Lu, MA-mamba: Multi-agent reinforcement learning with state space model, in: International Conference on Neural Information Processing, Springer, 2024, pp. 198–210.
[46] R.E. Kalman, A new approach to linear filtering and prediction problems, 1960.
[47] M.D. Shuster, Kalman filtering of spacecraft attitude and the QUEST model, J. Astronaut. Sci. 38 (3) (1990) 377–393.
[48] M.C. VanDyke, C.D. Hall, J.L. Schwartz, Unscented Kalman filtering for spacecraft attitude state and parameter estimation, AAS Publications Office Tucson, AZ, USA, 2004.